VASP Workflow

User Manual

Browser GUI · CLI Agents · SLURM HPC · High throughput · Diagnosis & repair

Keivan Esfarjani

v2.5 · October 2026 | https://github.com/KeivanS/VASP-Flow

Contents

I. Getting started 2

1. What VASP-Flow does and where to find what 2
1.1. Find what you need . . . 2
1.2. Key features . . . 2
1.3. Requirements . . . 3

2. Installing and configuring VASP-Flow 3
2.1. default_paths — platform configuration . . . 3
2.2. Configuring a cluster without the GUI . . . 4

II. Single calculations 5

3. Running a calculation from the browser (GUI) 5
3.1. Starting the GUI . . . 5
3.2. Three-tab workflow . . . 5
3.3. Setup tab — field guide . . . 5
3.3.1. Structure and POTCAR . . . 5
3.3.2. Functional . . . 6
3.3.3. Spin / SOC . . . 6
3.3.4. Other flags . . . 6
3.4. Task checkboxes . . . 6
3.5. Band structure options . . . 6
3.6. DOS options . . . 7
3.7. Workflow tab . . . 7
3.8. Results tab . . . 7
3.9. Viewing results of jobs run elsewhere . . . 7

4. Running a calculation from the command line 8
4.1. On a workstation: `vasp-agent.py` . . . 8

4.2. On a SLURM cluster: `vasp-agent-slurm.py` . . . 8
4.3. What the agent creates . . . 8

5. Describing the calculation: `instructions.txt` 8
5.1. An example to start from . . . 8
5.2. All keywords . . . 11

6. Choosing functional, spin, SOC and GGA+U 12
6.1. Functionals . . . 12
6.2. Spin-orbit coupling . . . 12
6.3. Magnetic materials (collinear spin) . . . 12
6.4. GGA+U . . . 13

7. Choosing the k-point mesh 13

8. Setting INCAR and KPOINTS tags yourself 14
8.1. Raw INCAR passthrough . . . 14
8.2. Explicit KPOINTS blocks . . . 14
8.3. Command-line INCAR overrides (`-incar`) . . . 15
8.4. Constant-pressure relaxation . . . 15

9. Band structures, DOS projections and ELF profiles 15
9.1. Band-structure k-path . . . 15
9.1.1. Auto-detection (recommended) . . . 15
9.1.2. Manual override . . . 16
9.2. Projected DOS . . . 16
9.3. ELF bond profiles . . . 16

10. Convergence tests for ENCUT and k-points 17

11. Cores, nodes, KPAR/NCORE and walltimes 18
11.1. KPAR and NCORE . . . 18
11.2. Cluster defaults: `profiles/slurm.json` . . . 18
11.3. Per-project resources (in `instructions.txt`) . . . 18
11.4. Walltime per step . . . 18

12. Wannier90, DFPT, phonons and LOBSTER 19
12.1. Wannier90 . . . 19
12.2. Born charges, dielectric and piezoelectric tensors (DFPT) . . . 19
12.2.1. Example . . . 19
12.3. Phonons (phonopy) . . . 20
12.4. LOBSTER bonding analysis (COHP / COBI / COOP) . . . 20

13. Worked examples 20
13.1. MoS_2 monolayer with SOC . . . 20
13.2. Nickel: magnetism + GGA+U . . . 21
13.3. GaAs: phonons with NAC . . . 21
13.4. High-pressure relaxation with custom INCAR tags . . . 21
13.5. Bi_2Se_3: topological insulator . . . 22

III. High-throughput screens 23

14. Running a high-throughput screen step by step 23
14.1. Step by step on the cluster (Rivanna) . . . 23

15. Scripts inside a high-throughput directory 24

IV. Checking and repairing runs 25

16. Finding and repairing failed runs (`vasp_diagnose.py`) 25
16.1. Quick start . . . 25
16.2. A worked example . . . 26
16.2.1. The list . . . 26
16.2.2. Step 1: diagnose . . . 26
16.2.3. Step 2: look at the result . . . 27
16.2.4. Step 3: submit . . . 27
16.2.5. Step 4: check again . . . 28
16.2.6. Materials held back for review . . . 28
16.3. Common situations . . . 28

17. Looking up one material or job quickly 29
17.1. A typical checking round . . . 30
17.2. `show_problems.sh`: what failed and what the fixes will do . . . 30
17.3. `skip_fixes.sh`: leave materials out . . . 30
17.4. `show_material_status.sh`: how far each step got . . . 30
17.5. `show_job_history.sh` and `show_job_log.sh`: what SLURM and the logs say . . . 31

18. Testing a fix on one material first 31

19. Maintenance tools for high-throughput directories 32

20. Troubleshooting: symptom and remedy 34
20.1. Setting up and running . . . 34
20.2. High-throughput runs on a cluster . . . 35
20.3. Quick OUTCAR checks . . . 35

Appendices: how it works 35

A. How the inputs are generated 35
A.1. k-mesh formula . . . 35
A.2. KPAR and NCORE rule . . . 36
A.3. Eigensolver (ALGO) of each step . . . 36
A.4. LOBSTER k-mesh and memory . . . 37
A.5. Relaxation → SCF hand-off . . . 37
A.6. Band-path detection . . . 37
A.7. Two-step ELF (`ELF: separate`) . . . 38
A.8. Convergence tests: test structure and parameter choice . . . 38
A.8.1. Shifted test structure . . . 38
A.8.2. Automatic parameter selection . . . 38
A.9. DFPT: fixed parallel and symmetry settings . . . 39
A.10. LOBSTER projected DOS vs. the VASP DOS . . . 39

B. How SLURM steps continue past the time limit **39**

C. High-throughput drivers: all options **40**
C.1. `ht-mp-scf.py` in detail 40
C.2. `ht-semimetals.py` in detail 41

D. How `vasp_diagnose.py` works **42**
D.1. Command line 42
D.2. The list file 43
D.3. Kinds of runs 43
D.4. When is a step OK? 44
D.5. Failure causes and their repairs 45
D.6. What is redone 46
D.7. Order of the resubmitted jobs 46
D.8. Output files 46
D.9. Limitations 47

E. Version history **48**

Part I.
Getting started

1. What VASP-Flow does and where to find what

VASP Workflow is a framework for setting up, running, and analysing DFT calculations with VASP. Two interfaces are provided:

- **Browser GUI** (`vasp-gui.py`) — a Flask web application: configure calculations through a form, run steps from the browser, view band-structure and DOS plots in real time.
- **CLI agents** (`vasp-agent.py` for workstations, `vasp-agent-slurm.py` for SLURM clusters) — read POSCAR and a plain-text `instructions.txt` file and generate all INCAR, KPOINTS, POTCAR, and job scripts automatically.

Both interfaces share the same `modules/` back-end.

On a cluster, `vasp_diagnose.py` and the scripts in `ht_tools/` check finished runs, say why a step failed, write the repairs and resubmit them (Part IV).

1.1. Find what you need

I want to …	Go to
install VASP-Flow and tell it where VASP and the POTCARs are	Section 2
set up and run one material with a form in the browser	Section 3
set up and run one material from the command line or on a cluster	Section 4
write or change `instructions.txt`	Section 5
choose the functional, spin, SOC or GGA+U	Section 6
change the k-point mesh	Section 7
set any INCAR or KPOINTS tag myself	Section 8
get band structures, DOS, ELF, phonons, Born charges, Wannier functions or LOBSTER bonds	Sections 9–12
set cores, nodes, KPAR/NCORE and walltimes	Section 11
run hundreds of Materials Project IDs on a cluster	Section 14.1
find out why runs failed and resubmit them	Section 16
check one material or one job quickly	Section 17
try a different INCAR setting on one material before resubmitting	Section 18
fix something that went wrong	Section 20
know how a value was chosen (k-mesh, KPAR/NCORE, ALGO, walltime continuation, …)	Appendix A

1.2. Key features

- Functionals: PBEsol, PBE, R2SCAN, HSE06, RVV10, LDA, AM05
- Spin-orbit coupling (SOC) and collinear spin (ISPIN=2)
- Magnetic materials: correct MAGMOM per atom; spin-resolved band and DOS plots (spin-up / spin-down)

- GGA+U (Dudarev, LDAUTYPE=2) per element and orbital
- Automatic k-path detection via spglib (Setyawan–Curtarolo standard paths), with a geometric fallback
- Convergence testing: ENCUT and k-mesh sweeps
- DOS with orbital projections via `sumo`
- Phonons via phonopy, Wannier90 interface, DFPT Born charges
- LOBSTER bonding analysis (COHP/COBI/COOP) via the `08_lobster` step
- Smart KPAR/NCORE auto-defaults scaled with MPI task count
- SLURM workflow with dependency-chained job submission

1.3. Requirements

Package	Notes
VASP 5.4+	`vasp_std`, `vasp_ncl` (SOC), optionally `vasp_gam`
POTCAR library	PAW pseudopotentials in per-element folders
Python 3.8+	Conda environment recommended
Flask	`pip install flask` — GUI only
sumo $\geq$ 2.0	`pip install sumo` — band and DOS plots
MPI	`mpirun`, `mpiexec`, or `srun`
phonopy	optional; `conda install -c conda-forge phonopy`
wannier90.x	optional; `conda install -c conda-forge wannier90`

2. Installing and configuring VASP-Flow

```
# 1. Clone the repository
git clone https://github.com/KeivanS/VASP-Flow.git
cd VASP-Flow

# 2. Install Python dependencies
pip install flask sumo

# 3. Optional: phonons and Wannier support
conda install -c conda-forge phonopy wannier90

# 4. First-time configuration (creates default_paths from template)
make setup
nano default_paths      # set VASP paths, POTCAR directory, MPI
    command
```

2.1. default_paths — platform configuration

The file `default_paths` (Makefile key=value syntax) is read at startup and its variables are exported as environment variables that the GUI uses as initial defaults. Settings saved via the GUI *System Setup* page take precedence; `default_paths` acts as the fallback for new installs or CLI-only use.

```
PYTHON          = python3
VASP_STD        = /path/to/vasp_std
VASP_NCL        = /path/to/vasp_ncl
VASP_GAM        = /path/to/vasp_gam       # optional
VASP_POTCAR_DIR = /path/to/potpaw_PBE
MPI_LAUNCH      = mpirun -np              # or: srun
MPI_NP          = 16
WANNIER90_X     = wannier90.x
LOBSTER_X       = lobster                 # lobster binary or full
   path
VISUALIZER_X    = ~/bin/jmol             # Jmol, VESTA, OVITO, ...
```

Three-layer configuration precedence (highest wins):

1. GUI System Setup page → saved to `settings.json`; also auto-synced to `profiles/slurm.json` for SLURM agents.
2. `default_paths` → Makefile bootstrap, read on every `make run`.
3. Built-in code defaults.

`profiles/workstation.json` is used only when running `vasp-agent.py --profile workstation` directly from the command line without the GUI.

2.2. Configuring a cluster without the GUI

When running on a remote supercomputer where the browser GUI is not available, executable paths and MPI settings must be configured directly in `profiles/slurm.json` (for the SLURM agent) or `profiles/workstation.json` (for the workstation agent).

```
# On the cluster, after cloning VASP-Flow edit the file:
nano profiles/slurm.json
# or
vi profiles/slurm.json
```

The relevant keys (to be edited) are:

```
{
  "vasp_std":    "/path/to/vasp_std",
  "vasp_ncl":    "/path/to/vasp_ncl",
  "vasp_gam":    "/path/to/vasp_gam",
  "wannier90_x": "wannier90.x",
  "lobster_x":   "/path/to/lobster",
  "mpi_cmd":     "srun",
  "mpi_np":      128,
  "modules":     ["intel/2023", "vasp/6.4"],
  "slurm": {
    "partition": "standard",
    "nodes":       2,
    "ntasks_per_node": 64,
    "time":        "12:00:00",
    "account":     "your-project"
  }
}
```

The `modules` list is prepended as `module load ...` lines in every generated `run.sh`. Alternatively, all path keys can be set as environment variables in your `.bashrc` (`VASP_STD`, `LOBSTER_X`, etc.) and the profile values left as empty strings to inherit them.

Note: `profiles/slurm.json` is auto-regenerated whenever you click *Save Settings* in the GUI, so the authoritative source for GUI users is the System Setup page. On a headless cluster, edit the JSON file directly and leave it in place.

Part II.
Single calculations

3. Running a calculation from the browser (GUI)

3.1. Starting the GUI

```
make run            # launches on http://localhost:5001
```

On a SLURM cluster — run on the login node, open on your laptop via SSH port-forwarding:

```
# On your laptop:
ssh -L 5001:localhost:5001 username@cluster

# On the cluster login node:
make run

# Open http://localhost:5001 in your laptop's browser
```

3.2. Three-tab workflow

1. **Setup** — configure the project, then click *Generate Workflow*.
2. **Workflow** — run each calculation step; live VASP output streams to the browser.
3. **Results** — band-structure and DOS plots; OUTCAR analysis.

To return to a previous project use the *Select project* dropdown. → **Workflow** jumps to the run tab; **Edit & Regenerate** re-opens Setup with all settings pre-filled.

3.3. Setup tab — field guide

3.3.1. Structure and POTCAR

Field	Description
Project name	Folder created in the working directory.
POSCAR	Paste the crystal structure, or click *Load file*. Click *Visualize* to convert the current POSCAR to XYZ and open it in the structure viewer configured under System Setup (Jmol, VESTA, OVITO, . . .).
POTCAR directory	Path to your PAW library. If an element has multiple variants (`Ga` vs `Ga_d`) a chooser appears.

3.3.2. Functional

Choice	Description
PBEsol	Optimised for solids; recommended starting point.
PBE	Standard GGA; widely used.
R2SCAN	Meta-GGA; good accuracy/cost balance.
HSE06	Hybrid — accurate band gaps, significantly slower.
RVV10	van der Waals; for layered or molecular materials.
LDA / AM05	Rarely needed.

3.3.3. Spin / SOC

Choice	Effect
None	Non-magnetic.
Collinear spin (ISPIN=2)	Sets `ISPIN = 2`; reveals the *Magnetic moment / atom* field (Section 6.3).
SOC ∥ z/x/y	Sets `LSORBIT`, `LNONCOLLINEAR`, `SAXIS`; uses the `vasp_ncl` binary.

3.3.4. Other flags

Field	Effect
Hexagonal BZ	Correct M point for hexagonal structures.
2D / slab	Sets $k_z = 1$ (Gamma-only out-of-plane).
GGA+U	Adds Hubbard U rows per element/orbital (Section 6.4).
Execution profile	Select *SLURM HPC* to generate `sbatch` scripts; edit `profiles/slurm.json` first.
MPI tasks	CPU cores per calculation.

3.4. Task checkboxes

Task	Directory	Notes
Structure relaxation	`01_relax`	IBRION=2; run first if geometry is not pre-optimised.
SCF	`02_scf`	Required before bands or DOS.
Band structure	`03_bands`	Line-mode KPOINTS; Section 9.1.
DOS	`04_dos`	Dense k-mesh; orbital projections via sumo.
DFPT	`06_dfpt`	Born charges, dielectric + piezoelectric tensors; Section 12.2.
Phonons	`07_phonons`	Phonopy finite-displacement; DFPT needed for NAC.
Wannier90	`05_wannier`	NSCF interface; Section 12.1.
LOBSTER	`08_lobster`	COHP/COBI/COOP bonding analysis; needs SCF; Section 12.4.

Note. Normal execution order: **relaxation** → **SCF** → **bands** → **DOS**. Each step copies POSCAR/CHGCAR from the previous one automatically.

3.5. Band structure options

Leave the *k-path* field blank to trigger **automatic detection** from the POSCAR lattice vectors (Section 9.1). Enter a path manually to override, e.g. `G-M-K-G`.

3.6. DOS options

Add projection rows to request orbital-resolved DOS:

Element: `Mo` Orbitals: **d** Element: `S` Orbitals: **p**

For spin-polarised calculations sumo plots spin-up positive and spin-down mirrored below zero automatically.

3.7. Workflow tab

Each step shows three controls: **Run** (submit the step), **Edit files** (open INCAR/KPOINTS/POSCAR in the browser; a `.bak` is created on first edit), and a **status badge**. **Run All Steps** chains every step and stops on failure.

3.8. Results tab

Panel	Contents
Band structure	Spin-polarised: blue = ↑, red dashed = ↓, same k-axis, $E_F = 0$. Non-magnetic: single-colour sumo plot. If sumo fails (e.g. a pymatgen gap-detection crash) the GUI falls back to a built-in `vasprun.xml` plotter so a band plot still renders.
Cumulative DOS	Stacked filled-area by element. Spin-polarised: up positive, down mirrored.
Projected DOS	Orbital-resolved via sumo; spin-down negated automatically.
OUTCAR analysis	Energy, Fermi level, magnetic moment, forces.

3.9. Viewing results of jobs run elsewhere

The GUI can show the results of calculations it did not set up or run — a finished cluster job, a whole high-throughput screen, or the small tarball from `collect_results.sh`. On the bar under the tabs, type a path in **Open results folder** and press **Open**:

Path	What is opened
one calculation, e.g. `.../materials/mp-8`	that project (listed as `mp-8`)
a highthroughput folder, e.g. `.../highthroughput_test`	every `materials/<id>` (listed as `highthroughput_test~mp-8`, ...)
an unpacked `screen_results.tar.gz` (`.../results`)	every `results/<id>`

The opened calculations are added to the project list (marked *opened*) and remembered in `settings.json`; **Results** shows the selected one. Plots that `analyze.sh` already wrote to `analysis/` are shown as they are; any that are missing (because `analyze.sh` never ran, or ran without matplotlib) are drawn by the GUI from the raw outputs — bands from `03_bands/EIGENVAL`, COHP / COBI / COOP and LOBSTER DOS from `08_lobster/*.lobster`, ELF from `02_scf/ELFCAR` — and saved into `analysis/`. Nothing is submitted or rerun. A step counts as done when its OUTCAR shows VASP finished (a relaxation must also reach the ionic criterion). **Forget opened** removes them from the list without touching any file.

`collect_results.sh` keeps this folder layout (`02_scf`, `03_bands`, `08_lobster`, `analysis` per material) with trimmed OUTCARs, so the tarball opens directly; set `FULL_OUTCAR=1` to keep the complete OUTCARs.

4. Running a calculation from the command line

4.1. On a workstation: `vasp-agent.py`

The vasp agents read by default the POSCAR file and the instructions.txt file which contains the project name and type of calculation the user wants. It creates a folder by than name, from which the calculations must be launched. Plots can be generated by running analyze.sh script from that folder.

```
./vasp-agent.py
./vasp-agent.py -i my_inst.txt -s struct.vasp

cd ProjectName
./run_all.sh          # runs every step in sequence
./analyze.sh          # plots after all steps complete
```

4.2. On a SLURM cluster: `vasp-agent-slurm.py`

```
./vasp-agent-slurm.py                  # uses profiles/slurm.json
./vasp-agent-slurm.py --profile slurm

cd ProjectName
./submit_all.sh        # sbatch with dependency chaining
squeue -u $USER        # monitor jobs
./analyze.sh           # run locally after jobs finish
```

Two-phase workflow when convergence tests are requested:

```
./submit_convergence.sh     # STEP 1 -- submit convergence jobs
# review results to decide on what to set for ENCUT and KPOINTS,
    then:
./submit_calculations.sh    # STEP 2 -- prompts ENCUT/k-mesh, submits
```

4.3. What the agent creates

```
ProjectName/
  instructions.txt
  POSCAR
  POTCAR               built once from $VASP_POTCAR_DIR
  01_relax/            INCAR  KPOINTS  POSCAR  POTCAR  run.sh
  02_scf/
  03_bands/
  04_dos/
  08_lobster/          if LOBSTER task selected (ISYM=0 NSCF +
     lobster)
  analysis/            plots generated by analyze.sh
  run_all.sh           workstation sequential runner
  submit_all.sh        SLURM job submission (dependency chained)
  analyze.sh           post-processing: plots + OUTCAR extraction
```

Without the GUI on a cluster, first set the VASP paths and SLURM defaults in `profiles/slurm.json` (Section 2.2).

5. Describing the calculation: `instructions.txt`

5.1. An example to start from

```
Project: My material name

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Fe-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Fe-d and O-p)

Convergence: Test k-points 6x6x6 to 12x12x12
             ENCUT 400-600 eV

MAGMOM: 4.0
NSW: 100
EDIFFG: -0.01
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 12:00:00
```

The parser is case-insensitive and searches the entire file, so section order is flexible.

5.2. All keywords

Keyword	Meaning
`Project:`	Project name; becomes the output folder.
`MAGMOM:`	Initial magnetic moments in μ_B: one number for all atoms (`4.0`, `-3.0`), per element (`Fe=4.0, O=0.6`) or a per-atom list (`2*4.0 2*-4.0`). Any `MAGMOM` line switches spin polarisation on; default 2.0 when only "collinear magnetization" is given (Section 6.3).
`ISIF:`	Relaxation type: 3=full, 2=ions only, 4=fixed volume.
`NSW:`	Max ionic steps. Default 100.
`EDIFFG:`	Force criterion in eV/Å. Default −0.01.
`ENCUT:`	Manual plane-wave cutoff in eV.
`KMESH:`	Explicit Gamma k-mesh overriding the automatic density, e.g. `8 8 8` or `12` (cubic). Applies to every automatic mesh (relax/SCF/DOS); for 2-D give the third value (`12 12 1`).
`KMESH_DENSITY:`	Automatic k-mesh density tier: `coarse` (1000 kpra, default for relaxation and convergence tests) or `fine` (6000 kpra, default for SCF/DOS/spectroscopic steps). kpra = $N_k \times N_{\text{atoms}}$ where $N_k = N_x N_y N_z$; subdivisions scale as $N_i \propto \lvert\mathbf{b}_i^*\rvert$. An integer is taken as the kpra target itself (also `KPRA:`, `KPOINTS_DENSITY:`). Overridden by explicit `KMESH` and by `KPOINTS` blocks.
`RELAX_KMESH_DENSITY:`	Density of the relaxation mesh (`coarse`, `fine` or an integer); default `coarse` whatever the SCF uses.
`PRESSURE:`	External pressure for constant-P relaxation, e.g. `10 GPa` or `50 kbar`. Forces `ISIF=3`, `IBRION=2`, `PSTRESS` (Section 8.4).
`INCAR: ... END_INCAR`	Block of literal INCAR tags merged into the generated INCAR(s); optionally per-step (Section 8.1).
`NKPTS:`	k-points per band segment. Default 40.
`MPI:`	Total MPI tasks (nodes × cores/node).
`KPAR:`	Global k-point parallelisation.
`NCORE:`	Global cores-per-orbital.
`SCF_KPAR:` *etc.*	Per-step KPAR: also BANDS_, DOS_, RELAX_, PHONONS_ (DFPT is forced to KPAR = NCORE = 1).
`SCF_NCORE:` *etc.*	Per-step NCORE; same prefixes.
`NODES:`	Compute nodes (SLURM).
`NTASKS_PER_NODE:`	MPI tasks per node (SLURM).
`PARTITION:`	SLURM partition.
`WALLTIME:`	One time limit for every step, e.g. `12:00:00` (default: per step, Section 11.4).
`RELAX_WALLTIME:` *etc.*	Per-step time limit (RELAX, SCF, BANDS, DOS, WANNIER, DFPT, PHONONS, LOBSTER).
`ACCOUNT:`	SLURM account / allocation.
`DFPT_EDIFF:`	EDIFF for DFPT. Default `1E-8`.
`PHONONS_DIM:`	Supercell, e.g. `2 2 2`.
`PHONONS_MESH:`	q-mesh, e.g. `20 20 20`.
`PHONONS_DISP:`	Displacement in Å. Default 0.01.
`PHONONS_BAND:`	High-symmetry path for phonon bands.
`PHONONS_NAC:`	`FALSE` to disable NAC.
`WANNIER_NUM_WANN:`	Number of Wannier functions.
`WANNIER_PROJ:`	Projections, e.g. `Ga:s;As:p`.
`WANNIER_EWIN:`	Energy window, e.g. `-5.0 to 15.0`.
`LOBSTER_MAX_KPTS:`	Largest $N_1 N_2 N_3$ of the symmetry-off LOBSTER mesh (default 10000, 0 = no limit).
`LOBSTER_MAX_KPW:`	Memory budget of the symmetry-off LOBSTER NSCF: largest k-points × plane waves; a denser mesh is reduced (default 6×10^6, 0 = no cap; Appendix A.4).

6. Choosing functional, spin, SOC and GGA+U

6.1. Functionals

```
Methods: PBEsol functional     # recommended for solids
Methods: R2SCAN functional     # meta-GGA
Methods: HSE06 functional      # hybrid -- more accurate but slower
Methods: RVV10 functional      # van der Waals
```

Default ENCUT values: PBE 400 eV, PBEsol 450 eV, R2SCAN 680 eV, HSE06 400 eV, RVV10 450 eV.

6.2. Spin-orbit coupling

```
Methods: SOC with magnetization in z-direction
```

Uses `vasp_ncl`. Sets `LSORBIT`, `LNONCOLLINEAR`, and `SAXIS`.

6.3. Magnetic materials (collinear spin)

```
Methods: collinear magnetization
MAGMOM: 4.0
```

Sets `ISPIN = 2` and `MAGMOM = N * value` in every INCAR, where N is the atom count from the POSCAR. Three forms of the `MAGMOM:` line are understood (any of them switches spin polarisation on, with or without the words "collinear magnetization"):

```
MAGMOM: 4.0                 # uniform; negative values keep their sign
MAGMOM: Fe=4.0, O=0.6       # per element (unlisted elements start at 0)
MAGMOM: 2*4.0 2*-4.0        # per atom in POSCAR order, n*value
    shorthand
```

A per-atom list must have exactly N entries; the agent stops with an explanatory message rather than silently falling back to a default.

Tip. Typical starting moments: Fe, Co $\approx$ 4–5 μ_B; Ni $\approx$ 2 μ_B; Cr, Mn $\approx$ 3–4 μ_B; rare earths $\approx$ 7 μ_B. VASP converges the self-consistent value automatically.

Antiferromagnets — give the alternating signs directly, e.g. `MAGMOM: 4.0 -4.0 4.0 -4.0`.

With SOC (`Methods: SOC with magnetization in x-direction`) the moments are $3N$ components in the non-collinear `MAGMOM`; the same `MAGMOM:` values are rotated onto the requested direction (without this VASP would start every moment along its default $(1, 1, 1)$). A list of exactly $3N$ numbers is used as the raw x, y, z components.

Spin-resolved plots produced automatically after DOS/bands run:

- *Cumulative DOS* — spin-up above the x-axis; spin-down mirrored below zero; elements colour-coded; solid = ↑, dashed = ↓.
- *Band structure* — spin-up in blue, spin-down in red dashed, same k-axis, $E_F = 0$.
- *Projected DOS* — sumo natively negates the spin-down channel in the orbital-resolved PDF.

Check the converged moment:

```
grep "magnetization␣(x)" OUTCAR | tail -5
```

6.4. GGA+U

```
Methods: GGA+U with U=4.0 on Fe-d orbitals
         GGA+U with U=3.0 on Ni-d orbitals
         GGA+U with U=6.0 on Ce-f orbitals
```

Dudarev scheme (LDAUTYPE=2); $U_{\text{eff}} = U - J$ with $J = 0$ set automatically. In the Dudarev formalism the parameter labelled U is already U_{eff}, so no separate J entry is needed. Combine freely with `collinear magnetization`.

Default values come from `hubbard_u_defaults.csv` (repo root: element, orbital, $U_{\text{eff}} = U - J$, ...; 3d, 4d, 5d, 4f, 5f elements) — the only source of default values (there are no built-in ones). Dudarev's scheme uses U_{eff} only, so the INCAR gets `LDAUU` $= U_{\text{eff}}$ and `LDAUJ` $= 0$ (a J column in the file is ignored); rows with $U_{\text{eff}} = 0$ (Zn, Cd) get no correction. These values were fitted for oxides, chalcogenides and halides, so their use is set by one optional flag in `instructions.txt`:

Flag	Effect
(none)	**default**: GGA+U on the tabulated d/f elements only if the POSCAR contains a chalcogen or halogen (O, S, Se, Te, F, Cl, Br, I); otherwise $U = 0$.
`GGA_U: ON`	tabulated U_{eff} on every tabulated element, whatever the compound.
`GGA_U: OFF`	no U (also `no GGA+U`).
`GGA_U: AUTO`	same as no flag.
`GGA+U with U=... on El-orb`	explicit values; always win.

The key may also be written `GGA-U`, `GGA+U` or `GGAU`, and key and value are case-insensitive (`gga+u: on`); likewise the command-line option of the high-throughput drivers accepts `--gga_u`, `--gga-u`, `--gga+u` (or upper case) with `auto`, `on`, `off` in any case.

The same U block is written to every step (relax, SCF, bands, DOS, LOBSTER, ...), so all steps share one Hamiltonian. Never automatic under R2SCAN/HSE06. All three routes honour the flag: the GUI (Setup, *Default Hubbard U*: auto / on / off), `vasp-agent.py` and `vasp-agent-slurm.py`; the high-throughput drivers write it with `--gga_u auto|on|off`.

7. Choosing the k-point mesh

All Gamma-centred meshes are generated automatically from the POSCAR geometry. Two density tiers are available via the `KMESH_DENSITY` key (Section 5.2):

Tier	kpra	Typical use	Notes
`coarse`	1000	Relaxation, convergence tests	Agent k-point sweeps start here
`fine`	6000	SCF, DOS, spectroscopic steps	Default for all production steps

Each axis gets a number of points proportional to its reciprocal-lattice length, so the spacing is the same in all directions; the formula is in Appendix A.1. The first line of each generated KPOINTS reports the mesh and the density reached. Any integer may be given instead of `coarse`/`fine` (`KMESH_DENSITY: 3000`); the relaxation has its own density (`RELAX_KMESH_DENSITY`, default coarse).

DOS mesh: always $2N_x \times 2N_y \times 2N_z$ (the SCF ratios are preserved; a 2-D slab keeps $N_z = 1$), for accurate spectral features.

LOBSTER mesh: the SCF mesh. The LOBSTER NSCF runs with `ISYM`=0, so it pays for every k-point of the mesh, not the irreducible wedge. It is therefore **kept within limits automatically**: if the mesh has more than `LOBSTER_MAX_KPTS` points ($N_1 N_2 N_3$, default 10000, about 20×20×20) or its k-points times plane waves exceed `LOBSTER_MAX_KPW` (6×10^6, the

memory limit), it is reduced, keeping its ratios, until both hold; the KPOINTS comment says so (Appendix A.4). The DFPT mesh is forced to match the SCF mesh exactly.

An explicit `KMESH: N1 N2 N3` entry overrides the automatic density for all steps, and a `KPOINTS` block (Section 8.2) overrides everything for the step it names.

8. Setting INCAR and KPOINTS tags yourself

Any VASP keyword can be specified explicitly — nothing is hard-wired. Three routes, in increasing precedence: (1) an `INCAR: ... END_INCAR` block in the instructions file (global or per-step); (2) the agents' `-incar` command-line option; (3) direct editing — in the GUI every generated INCAR is editable per step (Workflow tab → *Edit files*, with an automatic `.bak` backup), and of course the files on disk can be edited before running. This section covers the first two plus **constant-pressure relaxation**.

8.1. Raw INCAR passthrough

Place literal INCAR lines — in ordinary VASP syntax — inside an `INCAR: ... END_INCAR` block in your `instructions.txt`. They are merged into the generated INCAR(s):

```
INCAR:                    # global block -- applies to every step
   LREAL  = Auto
   ALGO   = Fast
   NELMIN = 6
END_INCAR
```

A tag that the agent already writes is **overwritten in place** (so `ALGO = Fast` above replaces the default `ALGO = Normal`); any tag the agent does not normally emit is appended under a clearly labelled `# User INCAR overrides` section, so nothing is silently lost.

To target a single step, name it on the header line. Recognised steps are `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`, `lobster`:

```
INCAR scf:                # applies only to 02_scf
   EDIFF = 1E-8
   SIGMA = 0.10
END_INCAR
```

Note. Precedence: a *per-step* block beats the *global* block, and both beat the agent's generated defaults. This is the recommended way to set tags the parser has no dedicated keyword for (e.g. `LMAXMIX`, `ADDGRID`, `LVTOT`, `IVDW`, `AMIX`). Full-line `#` comments in the instructions file are ignored by the parser, so a task keyword inside a comment does not enable that task.

8.2. Explicit KPOINTS blocks

The same block syntax writes a complete KPOINTS file, *unchanged*, for a step:

```
KPOINTS scf:              # or KPOINTS_SCF:
   my SCF mesh            # first line = KPOINTS comment
   0
   Gamma
   9 9 5
   0 0 0
END_KPOINTS
```

Steps: `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`, `lobster`; an unnamed `KPOINTS:` block applies to every step except the band path. Explicit blocks (INCAR and KPOINTS alike) **override every other setting**: keywords, `KMESH`, the density tier and the automatic

KPAR/NCORE. Steps derived from the SCF mesh — DOS and LOBSTER (2×), DFPT and Wannier90 (1×) — follow an explicit SCF block. A marker file .explicit_kpoints is left beside such a KPOINTS so that the convergence phase-2 k-mesh patching never overwrites it. KPAR and NCORE written in an INCAR block also steer the companion value (e.g. KPAR = 2 on 16 ranks gives NCORE = 2). In the GUI both block kinds are entered under *Setup* → *Advanced — explicit INCAR / KPOINTS*.

8.3. Command-line INCAR overrides (-incar)

Both vasp-agent.py and vasp-agent-slurm.py accept a repeatable -incar option that injects explicit VASP tags at invocation time, without touching the instructions file:

```
./vasp-agent.py -i instructions.txt -s POSCAR \
    --incar "ALGO=Fast; NELMIN=6" \       # every step
    --incar "scf:NBANDS=64"               # 02_scf only
```

Multiple tags in one option are separated by ; (not commas — values such as MAGMOM may contain commas). An optional step prefix (relax/scf/bands/dos/wannier/dfpt/phonons/lobster) targets a single step. Command-line tags are applied *after* the instructions-file blocks, so they win on conflicts.

Note. Exceptions: physics-critical guards are re-imposed regardless of overrides — ISYM=2 and KPAR=NCORE=1 in 06_dfpt, and the symmetry-off ISYM in 08_lobster.

8.4. Constant-pressure relaxation

A single PRESSURE line turns the relaxation into a constant-pressure optimisation: the agent forces ISIF = 3 (cell shape + volume + ions) and IBRION = 2, and imposes the target pressure through PSTRESS.

```
PRESSURE: 10 GPa        # also accepted: "5 kbar", "constant
   pressure 8"
```

VASP's PSTRESS tag is in **kBar** (1 GPa = 10 kBar). The agent converts automatically; if no unit is given, **GPa is assumed**. The resulting 01_relax/INCAR reads, e.g.:

```
# Constant-pressure relaxation @ 10 GPa  (PSTRESS = 100 kBar)
IBRION = 2
ISIF   = 3
NSW    = 100
EDIFFG = -0.01
PSTRESS = 100   ! external pressure in kBar (10 GPa)
```

Tip. PSTRESS adds the PV term so the relaxed cell sits at the requested external pressure. Use a tight EDIFFG and a well-converged ENCUT; the Pulay stress error grows at high pressure, so verify with a higher ENCUT before trusting the equilibrium volume.

How the SCF starts from the relaxation (which files are copied, ISTART/ICHARG) is described in Appendix A.5; which eigensolver each step uses in Appendix A.3.

9. Band structures, DOS projections and ELF profiles

9.1. Band-structure k-path

9.1.1. Auto-detection (recommended)

Omit the path in instructions.txt or leave the field blank in the GUI:

```
Tasks: band structure          # no "along ..." -> auto-detect from
    POSCAR
```

The path is the standard Setyawan–Curtarolo path of the actual space group (found with spglib), or a geometric guess when spglib is missing; details and the table of default paths are in Appendix A.6.

Note. For **conventional** FCC or BCC cells (right angles, 4 or 2 atoms), auto-detection returns the cubic (cP) path. Use the **primitive** cell for an unfolded FCC/BCC band structure — the primitive FCC cell has angles ≈ 60° and is recognised automatically.

The detection route appears in the KPOINTS comment:

```
Line-mode KPOINTS  [spglib ... -- auto-detected]    # spglib route
Line-mode KPOINTS  [FCC -- auto-detected]           # geometric
    fallback
```

9.1.2. Manual override

```
Tasks: band structure along G-X-W-K-G-L
```

Label	Coords	Use	
G	(0, 0, 0)	all	Γ
M	(0.5, 0.5, 0)	cubic, tet	(0.5, 0, 0) in hexagonal
K	(1/3, 1/3, 0)	hexagonal	
R	(0.5, 0.5, 0.5)	cubic	
Z	(0, 0, 0.5)	general	
A	(0, 0, 0.5)	hex, tet	
L	(0.5, 0, 0.5)	hexagonal	
H	(1/3, 1/3, 0.5)	hexagonal	
S	(0.5, 0.5, 0)	orthorhombic	
W	(0.5, 0.25, 0.75)	**FCC primitive**	
P	(0.25, 0.25, 0.25)	**BCC primitive**	
N	(0, 0, 0.5)	**BCC primitive**	

9.2. Projected DOS

All of the following forms are accepted:

```
DOS (projected on Mo-d and S-p)
DOS with projections on Ni:d and Ni:s
DOS with projection on Fe-d and O-p
```

Orbital channels: `s`, `p`, `d`, `f`. Separator is either `-` or `:`. Sumo generates an orbital-resolved PDF in `analysis/`.

9.3. ELF bond profiles

If an SCF calculation writes an `ELFCAR` (electron localization function, requested with `LELF = .TRUE.` — the high-throughput driver sets this automatically), the utility `elf_bonds.py` plots ELF along each non-equivalent nearest-neighbour bond, sampled from one atom centre to the other:

```
./elf_bonds.py [ELFCAR] [--out PREFIX] [--nn-scale 1.2] [--npoints
    200]
```

It parses the `ELFCAR` grid directly (only numpy + matplotlib), groups bonds by element pair and length (rounded to 0.01 Å) to pick the non-equivalent ones, and interpolates ELF along each bond with periodic boundary conditions. Output is `FORMULA_elf_bonds.png/pdf` (the prefix defaults to the reduced formula, e.g. `MoSe2_elf_bonds.pdf`; override with `-out`) showing ELF vs. distance along the bond, with the ELF = 0.5 homogeneous-electron-gas reference marked. `analyze.sh` runs it automatically when `02_scf/ELFCAR` is present, and the GUI Results tab shows the same plot whenever the SCF wrote an `ELFCAR`.

Tip. Reading the profile: ELF is high (near 1) where electrons are paired and localized. A covalent bond therefore shows an ELF *maximum* in the bonding region *between* the atoms (e.g. Si–Si ≈ 0.95 at mid-bond), not at the nuclei. Because VASP builds the `ELFCAR` from the pseudo (valence) density, the value *at* an atom centre is a PAW artifact — often low, even ≈ 0 for heavy atoms — so read the bonding region, not the endpoints.

Note. VASP does *not* compute ELF for non-collinear / spin-orbit runs (`LSORBIT`/`LNONCOLLINEAR`); it prints `"WARNING: ELF not implemented for non collinear case"` and writes an all-zero `ELFCAR`. `elf_bonds.py` detects this and reports it rather than producing a blank plot — re-run the SCF *without* SOC to get ELF.

ELF without slowing down the SCF. `LELF` needs KPAR = 1. With `ELF: separate` (or `ELF: two-step`) in `instructions.txt` the SCF runs at full KPAR and a short KPAR = 1 restart in `02_scf/elf/` writes the ELFCAR (Appendix A.7); `ht-mp-scf.py` does this by default.

10. Convergence tests for ENCUT and k-points

```
# Range (auto-steps)
Convergence: Test k-points from 6x6x1 to 18x18x1    # 2D material
             Test k-points from 6x6x6 to 12x12x12   # 3D material
             ENCUT 400-600 eV

# Explicit mesh list
Convergence: Test k-points 6x6x3, 12x12x6, 18x18x9

# ENCUT: all of these are equivalent (step defaults to 50 eV)
Test ENCUT 350-650      Test ENCUT 350 650     Test ENCUT 350 to 650
Test ENCUT 350,650      Test ENCUT 350:50:650  # start:step:stop
```

The `start:step:stop` form is the only one that sets the step; the others sweep in 50 eV increments. A convergence line that cannot be parsed now emits a `[warn]` on stderr rather than silently generating no tests at all.

Convergence INCARs are trimmed to what the test actually reads — the total energy in `OUTCAR`. `LWAVE`, `LCHARG`, `LORBIT` and `LELF` are dropped, which avoids writing a full WAVECAR per point and, because `LELF` pins KPAR to 1, restores k-point parallelism for the sweep.

Results in `00_convergence/kpoints/` and `00_convergence/encut/`.

```
./run_convergence.sh          # workstation
./submit_convergence.sh       # SLURM
./analyze_convergence.sh      # plots: energy / pressure / forces
```

After the runs the smallest ENCUT and k-mesh that meet the force and stress criteria are written to `00_convergence/converged_params.txt` and offered as defaults for phase 2 (Appendix A.8).

11. Cores, nodes, KPAR/NCORE and walltimes

11.1. KPAR and NCORE

KPAR and NCORE are set automatically per step from the number of MPI ranks and the step's k-points (rule in Appendix A.2). To set them yourself:

```
MPI: 128        KPAR: 4        NCORE: 8

# Per-step (take priority over globals)
SCF_KPAR: 4     BANDS_KPAR: 1     DOS_KPAR: 8
SCF_NCORE: 8    BANDS_NCORE: 16   DOS_NCORE: 8
RELAX_KPAR: 2   PHONONS_KPAR: 1   # DFPT is always KPAR=1, NCORE=1
```

11.2. Cluster defaults: `profiles/slurm.json`

```
{
  "vasp_std": "vasp_std",  "vasp_ncl": "vasp_ncl",
  "mpi_cmd":  "srun",
  "modules":  ["intel/2021", "impi/2021", "vasp/6.3.2"],
  "slurm": {
    "partition":       "standard",
    "nodes":           2,
    "ntasks_per_node": 64,
    "time":            "12:00:00",
    "account":         "mygroup"
  }
}
```

11.3. Per-project resources (in `instructions.txt`)

```
NODES: 4     NTASKS_PER_NODE: 32     PARTITION: gpu     ACCOUNT:
   myallocation
RELAX_WALLTIME: 12:00:00      # one step
WALLTIME: 48:00:00            # every step (instead of the per-step
   defaults)
```

11.4. Walltime per step

Each step is its own SLURM job with its own time limit, so short steps get short limits (and better queue priority). Defaults:

Step	Walltime
`01_relax`	08:00:00
`02_scf`	04:00:00
`03_bands`, `04_dos`	02:00:00
`08_lobster`	04:00:00 (symmetry-off NSCF over the full mesh, plus the LOBSTER projection)
others	the profile time

Priority: `<STEP>_WALLTIME` > `WALLTIME` > these defaults > `profiles/slurm.json`. In `ht-semimetals.py` directories the same defaults sit in `env.sh` (`TIME_RELAX ...`; `retune.sh --time-relax ...`); that is the only place to change them — the per-material `instructions.txt` carry no walltime.

A step that reaches its limit is stopped cleanly 15 minutes before and continued automatically in a requeued job (Appendix B).

12. Wannier90, DFPT, phonons and LOBSTER

12.1. Wannier90

```
Tasks: Wannier90 wannierization

WANNIER_NUM_WANN: 8
WANNIER_PROJ: Ga:s;As:p
WANNIER_EWIN: -5.0 to 15.0
```

The agent generates `05_wannier/wannier90.win` with the NSCF k-mesh and projection block. After the NSCF run, execute `wannier90.x` from the `05_wannier/` directory.

12.2. Born charges, dielectric and piezoelectric tensors (DFPT)

The `DFPT` task creates `06_dfpt`, which computes Born effective charges Z^*, the macroscopic dielectric tensor, and the piezoelectric stress tensor from the converged SCF charge density.

Two routes are generated automatically depending on the method:

Method	Route
PBE, PBEsol, LDA, AM05 (no SOC)	Linear response: `IBRION=8`, `LEPSILON=.TRUE.`
SOC, HSE06, or R2SCAN	Finite field (PEAD): `IBRION=6`, `LCALCEPS=.TRUE.` — VASP's DFPT is not implemented for noncollinear magnetism, hybrids, or meta-GGAs. Requires an insulating system.

KPAR, NCORE and ISYM are fixed for this step (Appendix A.9).

`run.sh` executes `extract_born.py` after VASP, which writes:

- `BORN` — phonopy NAC format: electronic ε_∞ + Z^* per atom.
- `born_charges.txt` — human-readable summary shown in the GUI Results tab: electronic (ion-clamped) ε_∞, ionic contribution, total static $\varepsilon_0 = \varepsilon_\infty + \varepsilon_{\text{ion}}$; Z^* tensors labelled by element; an acoustic-sum-rule check ($\sum_{\text{ions}} Z^*$ should vanish — a large residual indicates an under-converged k-mesh or EDIFF); and the piezoelectric stress tensors e (C/m^2, clamped-ion, ionic, and relaxed-ion total; meaningful only for non-centrosymmetric crystals).

`DFPT_EDIFF` in `instructions.txt` tightens the electronic convergence (default `1E-8`).

12.2.1. Example

```
Project: GaAs
Methods: PBE functional
Tasks: relaxation, SCF, DFPT Born charges and dielectric
DFPT_EDIFF: 1E-8
MPI: 16
```

In the GUI, tick **Born Charges & Dielectric Constant (DFPT)** on the Setup tab (the DFPT EDIFF field is optional); after running `06_dfpt` from the Workflow tab, the full summary appears in the **Born charges** panel of the Results tab. With SOC, HSE06 or R2SCAN selected under Methods, the finite-field INCAR is generated automatically — no extra input is needed. For LO-TO splitting in phonon spectra, combine with the phonon task as in the GaAs worked example (Section 13.3).

12.3. Phonons (phonopy)

```
Tasks: phonons lattice dynamics

PHONONS_DIM: 2 2 2
PHONONS_MESH: 20 20 20
PHONONS_DISP: 0.01
PHONONS_BAND: G M K G A
PHONONS_NAC: FALSE      # disable non-analytic correction
```

Note. For the non-analytic correction (important for polar materials), include `DFPT` in your task list so Born effective charges are computed.

12.4. LOBSTER bonding analysis (COHP / COBI / COOP)

Adding `LOBSTER COHP/COBI analysis` (keywords: `lobster`, `cohp`, `cobi`, `coop`, `bonding`) to the task list creates `08_lobster`: a symmetry-off NSCF that reads `02_scf/CHGCAR`, followed by a run of the `lobster` binary.

- **ISYM=0 is required** — a symmetrised k-mesh gives unusable COHP/ICOHP values (wrong sign, errors above 100%). The generated INCAR sets it automatically. With SOC enabled, **ISYM=−1** is used instead: spin-orbit coupling breaks time-reversal symmetry, which ISYM=0 would still assume.
- The k-mesh is the SCF mesh, reduced automatically above 10000 points or when the symmetry-off run would not fit in memory (`LOBSTER_MAX_KPTS`, `LOBSTER_MAX_KPW`, Appendix A.4).
- `NBANDS` is auto-set to at least the LOBSTER basis-function count; `LMAXMIX` is chosen from the elements (6 for f, 4 for d, 2 for sp).
- The LOBSTER binary is set with `LOBSTER_X` in `site.env` (default `lobster-5.1.0-OSX`); it can be overridden at runtime via `$LOBSTER_BIN`.

```
Project: NaCl
Methods: PBE functional
Tasks: SCF, LOBSTER COHP/COBI analysis
MPI: 16
```

Run with `./vasp-agent.py` (builds and runs `02_scf` + `08_lobster`); the GUI shows COHP/COBI/COOP plots on the Results tab. To aggregate averaged COHP/COBI over many projects into `lobster_acohp_acobi.csv`, run `python3 lobster_postprocess.py`.

Why the LOBSTER projected DOS and the VASP DOS differ is explained in Appendix A.10.

13. Worked examples

13.1. MoS_2 monolayer with SOC

```
Project: MoS2 monolayer

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks: structure relaxation
```

```
       SCF calculation
       band structure
       DOS (projected on Mo-d and S-p)

Convergence: Test k-points 8x8 to 20x20
             ENCUT 400-600 eV

MPI: 64
```

The generator detects hexagonal (hP) and uses G-M-K-G-A-L-H-A.

13.2. Nickel: magnetism + GGA+U

```
Project: Nickel GGA+U

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Ni-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Ni-d and Ni-s)

MAGMOM: 2.0
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 08:00:00
```

With a primitive FCC POSCAR (angles $\approx 60°$) the generator detects cF and uses G-X-W-K-G-L-U-W. Band and DOS plots show both spin channels.

13.3. GaAs: phonons with NAC

```
Project: GaAs phonons

Methods: PBEsol functional

Tasks: SCF calculation
       DFPT Born charges and dielectric
       phonons lattice dynamics

PHONONS_DIM: 4 4 4
PHONONS_MESH: 20 20 20
PHONONS_BAND: G X W L G K

MPI: 64     WALLTIME: 24:00:00
```

13.4. High-pressure relaxation with custom INCAR tags

A constant-pressure cell relaxation at 10 GPa, combined with raw INCAR passthrough to tighten convergence and add tags the parser has no dedicated keyword for:

```
Project: Silicon 10GPa

Methods: PBEsol functional
```

```
Tasks: structure relaxation
       SCF calculation
       band structure

PRESSURE: 10 GPa          # constant-P relax -> ISIF=3, IBRION=2,
   PSTRESS=100

# Tags applied to every step
INCAR:
   LREAL  = .FALSE.
   ADDGRID = .TRUE.
   LMAXMIX = 4
END_INCAR

# Tighter electronic convergence for the relaxation only
INCAR relax:
   EDIFF  = 1E-8
   EDIFFG = -0.005
END_INCAR

ENCUT: 600
MPI: 64
```

The `01_relax/INCAR` is generated with `ISIF=3`, `PSTRESS=100`, the global tags appended, and `EDIFF`/`EDIFFG` overridden to the per-step values; `02_scf` and `03_bands` receive only the global block.

13.5. Bi_2Se_3: topological insulator

```
Project: Bi2Se3 topological

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Bi-p and Se-p)
       Wannier90 wannierization

Convergence: Test k-points from 8x8x4 to 16x16x8
             ENCUT 400-600 eV

WANNIER_NUM_WANN: 18
WANNIER_PROJ: Bi:p;Se:p
WANNIER_EWIN: -2.0 to 2.0
MPI: 128
```

Part III.
High-throughput screens

14. Running a high-throughput screen step by step

Two drivers turn a list of Materials Project IDs (one `mp-id` per line; blank lines and `#` comments are ignored) into VASP runs. Both download the **primitive** cell from MP and use the same agents and input generator as the rest of VASP-Flow.

	`ht-mp-scf.py`	`ht-semimetals.py`
Steps per material	`01_relax` (ISIF = 3) → `02_scf` + ELF → `08_lobster` (optional)	`01_relax` (ISIF = 3) → `02_scf` → `03_bands` → `08_lobster` (no ELF, no DFPT)
Runs on	workstation or SLURM	SLURM only
Jobs	one job per step, submitted by `runall.sh`	one job per step (`job.sbatch <step>`), chained by `submit_all.sh`
Walltime	per step (Section 11.4)	per step, `TIME_*` in `env.sh`
Resources	one setting for all materials	ENCUT (1.3× max ENMAX) and job size per material
Extras	element/Z table	throttled, resumable `submit_all.sh`; `status.sh`, `retune.sh`, `analyze_all.sh`, `collect_results.sh`; POTCAR manifest
Use for	structures to relax and characterise (ELF, bonding)	screening a large set for bands and bonding

14.1. Step by step on the cluster (Rivanna)

1. Every new login: update the code and set the environment. The drivers download from MP, so run them on the login node (internet access).

```
cd ~/VASP-Flow && git pull
source ~/vaspenv/bin/activate     # python env (see below)
export MP_API_KEY=your_key        # materialsproject.org/api
export VASP_POTCAR_DIR=/path/to/PAW_PBE
```

First time only: `python3 -m venv ~/vaspenv && source ~/vaspenv/bin/activate && pip install pymatgen mp-api spglib numpy matplotlib`.

2. Put the list in a file, e.g. `~/VASP-Flow/lists/mylist.txt`.

3a. Semimetal screen (`ht-semimetals.py`):

```
cd ~/VASP-Flow
python3 ./ht-semimetals.py --list lists/mylist.txt \
    --out highthroughput_mylist --api-key "$MP_API_KEY" \
    --kpra 8000 --single-node
cd highthroughput_mylist
vi env.sh                      # VASP module, LOBSTER_BIN,
   ANALYSIS_PYTHON
./retune.sh --time-lobster 06:00:00    # optional: change a step
   walltime
./submit_all.sh                # POTCARs + submit; re-run = resume
```

```
./status.sh                    # progress per step
# when finished:
./postprocess_all.sh           # -> screen_lobster_all.csv
./collect_results.sh           # -> screen_results.tar.gz
```

3b. Relax + SCF + ELF (`ht-mp-scf.py`). It writes the material folders into the directory it is run from, so use a work directory:

```
mkdir -p /scratch/$USER/ht_mylist && cd /scratch/$USER/ht_mylist
python3 ~/VASP-Flow/ht-mp-scf.py -l ~/VASP-Flow/lists/mylist.txt \
    --agent slurm --no-chain --nodes 1 --ntasks-per-node 40 \
    --account elmgroup --kpra 8000 --encut 520
./runall.sh      # build inputs, submit relax -> SCF+ELF -> LOBSTER
squeue -u $USER
```

Check `~/VASP-Flow/profiles/slurm.json` first: its `modules` list must load a working VASP on the cluster (on Rivanna `vasp/5.4.4` alone; extra compiler/MPI modules cause version swaps). Choose `--encut` as about 1.3× the largest ENMAX in the set (the ISIF = 3 relaxation needs it against Pulay stress). `--no-chain` lets the materials run in parallel; the default (chained) runs everything one after another, and one failed step then blocks all later jobs.

POSCARs already downloaded? Neither driver downloads a structure it already has. `ht-semimetals.py` reuses `<out>/_ht_inputs/<id>/POSCAR`, `ht-mp-scf.py` the `_ht_inputs/` of the current directory. To take them from another folder — e.g. an older high-throughput directory — add `--poscar-dir <folder>` (repeatable; it looks for `<id>/POSCAR`, `materials/<id>/POSCAR`, `_ht_inputs/<id>/POSCAR` or `<id>.vasp`). No MP key or internet is needed then; only missing structures are downloaded.

```
python3 ./ht-semimetals.py --list lists/mylist.txt --out
   highthroughput_new \
    --poscar-dir highthroughput_mylist --kpra 8000 --single-node
```

4. Test first. Run two or three materials before a long list, then read the timings and look at the plots (the per-step walltimes of Section 11.4 are defaults; steps that hit them continue automatically):

```
grep "Elapsed time" materials/*/0*/OUTCAR      # ht-semimetals
grep "Elapsed time" */0*/OUTCAR                # ht-mp-scf
```

5. Look at the results in the GUI (Section 3.9): unpack `screen_results.tar.gz` on your computer (or run the GUI on the cluster with `ssh -L 5001:localhost:5001`) and use **Open results folder**; missing plots are drawn there.

All options of the two drivers are listed in Appendix C. When the jobs have finished, check and repair the runs as described in Part IV.

15. Scripts inside a high-throughput directory

A directory made by `ht-semimetals.py` contains these scripts; run them from that directory.

Script (in the directory)	Purpose
`env.sh`	The one file to edit on the cluster: modules, `VASP_STD`, `LOBSTER_BIN`, `VASP_POTCAR_DIR`, `ANALYSIS_PYTHON`, step walltimes `TIME_*`, `VF_MAX_RESTARTS`, disk policy.
`submit_all.sh [N]`	Build missing POTCARs, submit each material's unfinished steps as a chain (at most `MAX_QUEUED`=200 jobs queued); skips finished/queued materials, so it is also the resume command.
`status.sh`	Per-step completion counts; writes `incomplete.txt`.
`retune.sh`	Change the step walltimes in `env.sh` (`--time-relax`, `--time-scf`, `--time-bands`, `--time-lobster`, or `--time` for all) or partition/account in every job script.
`make_potcars.sh`	Build and verify the POTCARs (run automatically).
`analyze_all.sh [id]`	Redo the plots (each job already runs `analyze.sh`).
`postprocess_all.sh`	Per-material and combined LOBSTER bond tables.
`collect_results.sh`	Small `screen_results.tar.gz` with the per-step layout (trimmed OUTCARs; `FULL_OUTCAR=1` for full).

Part IV.
Checking and repairing runs

16. Finding and repairing failed runs (`vasp_diagnose.py`)

After a batch of VASP-Flow calculations — one project or a few thousand materials — some runs will not have finished: a job ran out of memory or time, an SCF did not converge, a band structure diverged, LOBSTER stopped, a convergence test broke off. `vasp_diagnose.py` finds these runs, tells you why each one failed and writes, for each of them, a shell script that repairs the inputs and resubmits exactly what has to be redone. One more script submits all of these repairs.

It checks the steps `00_convergence`, `01_relax`, `02_scf`, `03_bands`, `04_dos`, `06_dfpt` and `08_lobster`, for runs made with `vasp-agent.py`, `vasp-agent-slurm.py` or the high-throughput drivers.

> **Tip.** Running `vasp_diagnose.py` never changes or submits anything. It only writes a report and scripts. Your runs are touched only when you run those scripts.

16.1. Quick start

On the cluster, in your VASP-Flow folder, give it the list file you made the runs from:

```
cd ~/VASP-Flow
python3 vasp_diagnose.py my_list.txt --out diag_round1    # 1.
    diagnose (read-only)
bash ht_tools/show_problems.sh diag_round1                # 2. what
    failed, what will change
nohup bash diag_round1/submit_all_fixes.sh > fixes1.log 2>&1 &    #
    3. resubmit
```

That is all. `my_list.txt` stands for your own list, e.g. `semimetals_list.txt`: the file of material names that the runs were created from, one per line. A name is whatever the material's

folder is called — a Materials Project ID (`mp-149`), a formula (`GaAs`), `GaAs_PBE`, ... The script finds each material's folder by itself. When the resubmitted jobs have finished, run step 1 again to check that everything is now fine.

> **Note.** Run it when your jobs have left the queue. Materials that still have jobs queued or running are skipped (shown as `queued`).

16.2. A worked example

Without `--out`, the output folder is named `vasp_diagnose_<date>_<time>`, as in this example.

Say you created 300 materials with a high-throughput driver from `~/VASP-Flow/semimetals_list.txt`, into the run folder `~/VASP-Flow/HT_semimetals/`, and the queue is now empty.

16.2.1. The list

The list is the file you already have, for example

```
# ~/VASP-Flow/semimetals_list.txt
mp-149
mp-2534
mp-20351
Bi2Se3
GaAs_PBE
...
```

Each name is looked up in the folder that holds the list file, so `mp-149` is found as `~/VASP-Flow/HT_semimetals/materials/mp-149`, and `Bi2Se3` as `~/VASP-Flow/HT_semimetals/materials/Bi2Se3` or as a project folder `~/VASP-Flow/Bi2Se3`. Names must match the folder names exactly (`GaAs` does not match `GaAs_PBE`). You can also list folder paths, or mix names and paths (section D.2).

16.2.2. Step 1: diagnose

```
cd ~/VASP-Flow
python3 vasp_diagnose.py semimetals_list.txt
```

It prints a summary — how many materials are fine and how many failed for which reason:

```
Folders checked: 300   (layouts: ht 300)
  ok              262
  oom              14
  scf_nelm          6
  timeout           5
  spilling_high     5
  lobster_fail      4
  nscf_diverged     2
  missing           1
  unknown           1
Fix scripts: 31 to submit, 1 to review (unknown cause)
  review:
     .../vasp_diagnose_20261012_1530/fix/HT_semimetals__mp-20351.sh
Report : .../vasp_diagnose_20261012_1530/diagnosis.tsv
Submit : nohup bash .../submit_all_fixes.sh >
    .../submit_all_fixes.log 2>&1 &
```

Here 262 materials are fine. 31 failed for a known reason and have a fix script ready. One failed for a reason the tool does not recognise; its script is kept back for you to look at. 5 finished but have a high LOBSTER charge spilling (reported only), and one name of the list has no folder at all (`missing`). Section D.5 explains each name.

Everything is written into a new folder `vasp_diagnose_<date>_<time>/`:

```
vasp_diagnose_20261012_1530/
  diagnosis.tsv          the report: one line per material
  fix/<name>.sh          one repair script per failed material
  submit_all_fixes.sh    runs all the repair scripts
```

16.2.3. Step 2: look at the result

`diagnosis.tsv` has one line per material: the status of every step and what will be done:

```
label                  ...  steps
                                          action
HT_semimetals__mp-149      relax:ok scf:ok bands:ok lobster:ok
             ok
HT_semimetals__mp-2534     relax:ok scf:oom bands:not_run
   lobster:not_run oom -> scf>bands>lobster
HT_semimetals__mp-8062     relax:ok scf:scf_nelm bands:ok lobster:ok
       scf_nelm -> scf>bands>lobster
HT_semimetals__mp-20351    relax:unknown scf:ok ...
                        REVIEW unknown -> relax>scf>bands>lobster
```

Read it with `bash ht_tools/show_problems.sh vasp_diagnose_20261012_1530`, which lists only the problem lines and what each fix changes (Section 17), or with `less -S` or a spreadsheet. Then open one or two repair scripts to see what they will do, e.g.

```
less vasp_diagnose_20261012_1530/fix/HT_semimetals__mp-2534.sh
```

The top of each script says what was found, including the error lines from the logs:

```
# vasp_diagnose fix for HT_semimetals__mp-2534
# folder : /home/me/VASP-Flow/HT_semimetals/materials/mp-2534
   (layout: ht)
# diagnosed 2026-10-12 15:30:  relax: ok  scf: oom  bands: not_run
   lobster: not_run
#   scf | forrtl: severe (41): insufficient virtual memory
# resubmits: scf -> bands -> lobster
```

and further down you see the changes and the submission:

```
set_tag 02_scf/INCAR KPAR 8                 # KPAR was 40
stash 02_scf                                # old outputs ->
   stale_<date>/02_scf/
...
J_scf=$(sbatch --parsable --job-name=mp-2534_scf --time=00:30:00
   --mem=0 job.sbatch scf)
J_bands=$(sbatch ... --dependency=afterok:$J_scf job.sbatch bands)
J_lobster=$(sbatch ... --dependency=afterany:$J_bands job.sbatch
   lobster)
```

You may edit a script before submitting it (another walltime, another KPAR, ...).

16.2.4. Step 3: submit

```
nohup bash vasp_diagnose_20261012_1530/submit_all_fixes.sh >
    fixes.log 2>&1 &
tail -f fixes.log
```

`fixes.log` shows one block per material:

```
== HT_semimetals__mp-2534
  scf: KPAR 40 -> 8 (40 ranks)
  submitted scf = 4512301 (00:30:00)
  submitted bands = 4512302 (00:30:00)
  submitted lobster = 4512303 (03:00:00)
```

`nohup` lets it keep running after you log out. It never puts more than 200 of your jobs in the queue at once (it waits until there is room). If it is interrupted, start it again: materials it already submitted are skipped.

16.2.5. Step 4: check again

When the queue is empty again, diagnose once more, into a new folder:

```
python3 vasp_diagnose.py semimetals_list.txt --out diagnose_round2
```

The repaired materials now show `ok`. The ones that failed again get a new diagnosis and a new script. Repeat until only `ok` (and cases you accept, such as `spilling_high`) are left.

16.2.6. Materials held back for review

Two kinds of scripts are held back: `unknown` causes, and failures whose usual repair is already in the INCAR (shown as `(fix already in INCAR)` — running the same repair again would fail the same way). For these, read the error lines at the top of the script. Then either repair the run by hand, or let the script run as it is: open `submit_all_fixes.sh`, where these scripts appear at the end as commented-out lines, and remove the `#`. To submit all unknown cases without editing, diagnose again with `--retry-unknown`.

16.3. Common situations

Only some runs, or a single project. Make a short list with just those material names, or give a folder:

```
echo GaAs_test > one.txt;  python3 vasp_diagnose.py one.txt
echo HT_semimetals > run.txt; python3 vasp_diagnose.py run.txt   #
    all materials of that run
```

Runs moved to another disk (e.g. scratch). If a run folder holds a `MOVED_TO_SCRATCH.txt` note (written by `ht_tools/resume_in_scratch.sh`), its new place is used automatically. Otherwise tell the script where to look:

```
python3 vasp_diagnose.py semimetals_list.txt --root
    /scratch/$USER/VASP-Flow
```

The same name in two runs. If a name is found in more than one run folder, all copies are checked and a line `!!  GaAs found in 2 runs` is printed. To check only one of them, put its path in the list instead of the name.

Projects with convergence tests. For a project made in two phases (convergence tests first, then `submit_calculations.sh` or `run_calculations.sh`), broken ENCUT/k-point tests are repaired and resubmitted; the production steps are *not* started for you, since you choose their ENCUT and k-mesh from the tests. Such a project shows `phase2_pending` once the tests are complete.

Local projects (no SLURM). Their repair script runs VASP directly, one step after the other. Run that script on a machine where VASP may run (not on a login node): `bash vasp_diagnose_*/fix/projects__GaAs_test.sh`.

Leave a material out. `bash ht_tools/skip_fixes.sh DIAG_DIR NAME ...` comments its line in `submit_all_fixes.sh` out (`--undo` puts it back), e.g. for a duplicate that another run replaces.

Fewer jobs in the queue. `MAX_QUEUED=50 nohup bash .../submit_all_fixes.sh > fixes.log 2>&1 &`

Stop submitting. `pkill -f submit_all_fixes.sh`. Jobs already submitted keep running; cancel them with `scancel` if needed.

Undo a repair. Nothing is deleted. Cancel the material's jobs, move the files from `<folder>/stale_<date>/<step>/` back into `<step>/` and remove the `TAG = value` lines added at the end of the INCAR. `01_relax/POSCAR.orig` is the structure before the first restart.

Problems in the log. `awk '/^!!!|^-- /' fixes.log` lists failed submissions (`!!!`) and materials skipped because they had jobs in the queue or their outputs had changed since the diagnosis (`--`); for the latter, diagnose again.

How the tool decides what failed and what to redo — options, the list file, the failure classes and their repairs, the outputs and the limitations — is in Appendix D.

17. Looking up one material or job quickly

Five short scripts in `ht_tools/` answer the questions that come up while checking runs, so that you do not have to remember long `grep`/`awk` commands. Run them from `~/VASP-Flow`. A material can be given by its name (`mp-20352`, `GaAs`) — it is looked up in `/scratch/$USER/VASP-Flow/*/materials/` (and `~/VASP-Flow/*/materials/`, skipping runs that were moved to scratch) — or as a folder path. A name found in two runs is shown for both. Started without arguments, each script prints how to use it.

Question	Command
Which materials failed, and what will each fix change?	`bash ht_tools/show_problems.sh diag_round3` (`--all`: also the `spilling_high` ones)
Leave some materials out of the resubmission	`bash ht_tools/skip_fixes.sh diag_round3 NAME ...` (`--undo` puts them back)
How far did each step of a material get?	`bash ht_tools/show_material_status.sh mp-20352 [relax scf bands dos lobster]`
What did SLURM do with its jobs?	`bash ht_tools/show_job_history.sh mp-20352 [mp-6430 ...] [-S 2026-10-01]`
What do the job logs say at the end?	`bash ht_tools/show_job_log.sh mp-20352 lobster [LINES] [JOBID]`

Only `skip_fixes.sh` changes a file (it comments lines of `submit_all_fixes.sh` in or out); the others only read.

17.1. A typical checking round

```
cd ~/VASP-Flow
python3 vasp_diagnose.py ht_list.txt --since 2026-10-01 --out
    diag_round4
bash ht_tools/show_problems.sh diag_round4                # what
    failed, what the fixes do
bash ht_tools/show_material_status.sh mp-20352 lobster    # a closer
    look at one material
bash ht_tools/show_job_log.sh mp-20352 lobster            # ... and
    at its last log
bash ht_tools/skip_fixes.sh diag_round4 HT_1-500__mp-6430   # do not
    resubmit this one
nohup bash diag_round4/submit_all_fixes.sh > fixes4.log 2>&1 &
```

17.2. `show_problems.sh`: what failed and what the fixes will do

Prints three blocks: every material of `diagnosis.tsv` that is not `ok` (label, per-step status, planned action); which fix scripts `submit_all_fixes.sh` will `RUN` and which are `skipped`; and, per fix script, the changes it makes (e.g. `lobster: nscf_diverged -> ALGO = Fast`, `scf: walltime 01:00:00`). Check this before every submission.

17.3. `skip_fixes.sh`: leave materials out

```
bash ht_tools/skip_fixes.sh diag_round4 HT_1-500__mp-6430
    HT_1-500__mp-7889
bash ht_tools/skip_fixes.sh diag_round4 --undo HT_1-500__mp-6430
```

Each name is matched as part of the fix script's name, so `mp-6430` skips the material in every run. It prints the new `RUN`/`skipped` list. Typical uses: a duplicate of a material that another run replaces, or a material whose result you accept as it is (e.g. a relaxation that stops just short of EDIFFG with tiny forces).

17.4. `show_material_status.sh`: how far each step got

For every step folder of the material (or only the steps named):

- a header: `finished` or `not finished`, and **how long VASP ran** — the `Elapsed time` of a finished OUTCAR, or, for an unfinished one, the time from VASP's start to the last write of OUTCAR. This is the real running time, not the walltime limit of the job;
- the INCAR tags that matter for failures (ALGO, KPAR, NCORE, NBANDS, NELM, NSW, EDIFF, EDIFFG, ISYM, ICHARG, ISTART, LDAU);
- **every ionic step** of OSZICAR with the last 5 electronic steps before it;
- for the relaxation: converged or not, the last max force and pressure;
- when VASP did not finish: when OUTCAR was last written and its last 10 lines, which show where VASP stopped (also when it never got to an electronic step).

```
== 01_relax (finished, ran 02:13:26)
   ALGO = Normal
   NSW = 100
   EDIFFG = -0.01
   ...
   DAV:   6    -0.310074960321E+02   -0.42113E-05   ...
   DAV:   7    -0.310074982731E+02   -0.15960E-05   ...
   DAV:   8    -0.310074981222E+02    0.15085E-06   ...
    100 F= -.31007498E+02 E0= -.31007498E+02  d E =0.675983E-02   (8
       electronic steps)
   relax NOT converged
   FORCES: max atom, RMS     0.032426    0.027855
   external pressure =       -0.12 kB  Pullay stress =        0.00 kB
```

Reading OSZICAR. Lines starting with `DAV:` (Davidson), `RMM:` (RMM-DIIS) or `CG:` are *electronic* steps: step number, total energy, its change dE, the change of the band energies, the number of $H\psi$ evaluations and the density residual. A line `N F= ... E0= ... d E = ...` closes *ionic* step N (one relaxation or MD step): `F` is the free energy, `E0` the energy extrapolated to zero smearing, `d E` the change from the previous ionic step (an MD run adds `T=` and `EK=`). An SCF, bands, DOS or LOBSTER run has one ionic line at the end. The status prints the number of electronic steps of each ionic step; if the run stopped in the middle of one it says `... ionic step N still running`.

17.5. `show_job_history.sh` and `show_job_log.sh`: what SLURM and the logs say

`show_job_history.sh` lists, for every job whose name contains one of the patterns, SLURM's record since `-S` (default: 3 days ago): job id, name, state (`COMPLETED`, `TIMEOUT`, `FAILED`, `OUT_OF_MEMORY`, `CANCELLED`), elapsed time, time limit, exit code and end time — and which of these jobs are in the queue now. It shows at a glance whether a step ran out of time, crashed after a minute (an input or memory problem) or was cancelled.

`show_job_log.sh` prints the last lines (default 15) of the newest `.out` and `.err` log of one step, or of the job id given. VASP's own error message is usually in `.out`, the crash trace of MPI in `.err`.

18. Testing a fix on one material first

When it is not clear which setting cures a failure, try a few on one material in short test jobs before changing the production run. Two scripts make test copies in `tests/` of the run folder — outside `materials/`, so the production chain, the diagnosis and the collection scripts never see them.

```
cd /scratch/$USER/VASP-Flow/HT_redo_plainRe
bash ../ht_tools/test_step.sh mp-7889 bands fast    "ALGO=Fast"
bash ../ht_tools/test_step.sh mp-7889 bands kpar4   "KPAR=4; NCORE=2"
bash ../ht_tools/test_step.sh mp-7889 bands ref     ""          #
   unchanged, for comparison
bash ../ht_tools/test_step.sh --report tests/mp-7889_bands_*    #
   while they run, and after
```

Script	What it tests
`test_step.sh ID scf|bands|lobster TAG "T=V; ..."`	The SCF, bands or LOBSTER NSCF of material `ID` in `tests/ID_STEP_TAG/`: the step's INCAR and KPOINTS, the material's POTCAR and POSCAR and, for bands/LOBSTER, `02_scf/CHGCAR` (no WAVECAR). Only VASP runs, not the LOBSTER program.
`test_relax.sh ID TAG "T=V; ..."`	The relaxation, from the original input cell, in `tests/ID_TAG/`.

Each `TAG=VALUE` replaces that tag in the INCAR (or adds it); `TAG=` with no value removes it (`"NSW=0; IBRION=-1; ISIF="` turns a relax into an SCF). KPAR is lowered to a divisor of the core count if needed. The tests are small and short so that they start soon: 40 cores and 1 hour by default, changed with `T_NTASKS=`, `T_TIME=`, `T_PARTITION=` in front of the command.

`--report` prints one line per test: status (`queued`, `running`, `ended`, `finished`, `timeout`), whether EDIFF was reached, the number of electronic steps, the first, last and largest $|E|$. A largest $|E|$ above $\sim 10^5$ eV means the run diverged. `test_relax.sh --report` gives energy, volume, pressure and max force per ionic step.

Tip. Once a setting works, put it into production: let `vasp_diagnose.py` write the fix (it knows the common ones), or edit the step's INCAR in `materials/ID/` yourself and resubmit that step, e.g. `cd materials/ID && sbatch -job-name=ID_bands -time=01:00:00 job.sbatch bands`. Delete `tests/` when done.

19. Maintenance tools for high-throughput directories

Besides the diagnosis (Section 16), the look-up tools (Section 17) and the test scripts (Section 18), `ht_tools/` holds scripts that act on a whole high-throughput directory. Run them from that directory:

```
cd /scratch/$USER/VASP-Flow/HT_1-500
bash ../ht_tools/audit_run.sh              # read-only
bash ../ht_tools/cleanup_large.sh          # dry run
bash ../ht_tools/cleanup_large.sh --apply # do it
```

All of them are dry runs unless `--apply` is given, skip materials with queued jobs, and never delete outputs (except `cleanup_large.sh`, whose job is to delete large files no step needs any more): they are moved into a `stale_<date>` folder of the material. Full descriptions are in `ht_tools/README.md`.

Script	Purpose
`audit_run.sh [BACKUP ...]`	Read-only check before adding a run to the database: per material missing / queued / relax not converged / SCF not finished or hit NELM / LOBSTER not finished, empty ICOHPLIST or spilling above `SPILL_MAX`; whether it is in a collected archive and in each backup given. Writes `audit_<date>.tsv`.
`collect_lobster.sh`	Tarball of the data the descriptor database needs (raw LOBSTER files, structures, EIGENVALs, trimmed OUTCARs, `_status.tsv`); incremental. `SCF_EIG_ONLY=1` packs only the SCF EIGENVALs of all finished materials.
`resume_in_scratch.sh`	Moves a run out of the home quota to `/scratch/$USER/VASP-Flow/<run>` (leaves `MOVED_TO_SCRATCH.txt` behind, which `vasp_diagnose.py` follows) and resubmits every unfinished material from its first unfinished step. Run under `nohup`.
`cleanup_large.sh`	Delete WAVECAR/CHGCAR no remaining step needs (`02_scf/CHGCAR` is kept unless `--deep`).
`reset_stale.sh`	Redo steps whose inputs changed (relax unconverged or finished after the SCF, truncated LOBSTER files, blank EIGENVAL).
`fix_failed.sh [ids]`	The older, high-throughput-only predecessor of `vasp_diagnose.py`: finds broken chains (a step FAILED / TIMEOUT / OUT_OF_MEMORY, the rest `DependencyNeverSatisfied`), repairs and resubmits them. Prefer `vasp_diagnose.py`.
`patch_relax_ibrion.sh`, `patch_lean_outputs.sh`	One-off patches of a directory generated before those fixes.

20. Troubleshooting: symptom and remedy

20.1. Setting up and running

Symptom	Solution
POTCAR not found	Check `VASP_POTCAR_DIR` in `site.env` or GUI System Setup. Use the POTCAR chooser for unusual variants (`Ga_d`, `Fe_sv`).
`vasp_std` not found	Set the full binary path in `site.env` or GUI System Setup.
WARNING: may not have converged	Increase NSW (ionic) or NELM (electronic) in INCAR and re-run.
Wrong bands ($W = \Gamma$)	Conventional cubic cell detected; use the *primitive* cell for FCC/BCC so auto-detection picks cF/cI.
Projected DOS missing	Check instructions.txt: `projected on` and `projections on` are both accepted. Verify element names match POSCAR.
KPAR error	KPAR × NCORE must divide total MPI tasks. Use auto-defaults or set explicitly valid values.
GUI unreachable	Check SSH tunnel: `ssh -L 5001:localhost:5001 user@cluster`.
Project missing from dropdown	Folder must contain `POSCAR` + `settings.json` in the directory where `vasp-gui.py` was started.
Band plot empty or not shown	Usually `sumo-bandplot` crashed on this band structure; the GUI now falls back to a built-in plotter automatically. An empty plot also means a wrong E_F: the Fermi level is read from `04_dos/OUTCAR` (then `02_scf/OUTCAR`), since `03_bands` eigenvalues share the NSCF/DOS energy reference.

20.2. High-throughput runs on a cluster

Symptom	What to do
Jobs stay in the queue as `DependencyNeverSatisfied`	A step before them did not end OK, so they can never start. `scancel` them, then run `vasp_diagnose.py` (Section 16); it finds the failed step and writes the repair.
Bands/DOS/LOBSTER energies run away to 10^6–10^{10} eV	Pure Davidson diverging at fixed density (seen for Ba_2YReO_6, Sr_2YReO_6). New runs use `ALGO = Fast` in these steps; `vasp_diagnose.py` sets it, then `VeryFast`, for older ones (Appendix A.3).
`forrtl: severe (41): insufficient virtual memory`, the job fails within a minute	Out of memory while setting up. The usual case is the LOBSTER NSCF: with `ISYM = 0` it keeps every k-point of the mesh (6916 for 24×24×24). Halve its mesh in `08_lobster/KPOINTS` (`vasp_diagnose.py` does this; new runs are capped from the start, Appendix A.4). More memory per rank (one node, fewer ranks) did not help. For other steps lower KPAR.
SCF reached EDIFF (the `1 F=` line is in OSZICAR) but the job still hit its time limit	VASP stalled after convergence, so more walltime will not help. Look at where it stopped (`show_material_status.sh ID scf` prints the OUTCAR tail and the time of its last write) and at the job log before resubmitting.
The relax ends `relax_unconv` in every round, with small forces	Check the last max force and pressure (`show_material_status.sh ID relax`). If they are close to EDIFFG and a few kB, accept the structure and leave the material out (`skip_fixes.sh`).
A material shows `queued` in two runs, although only one has jobs	Both runs use the same job name (`<material>_<step>`); diagnose again when the queue is empty.
A run folder in `~/VASP-Flow` looks empty	It was moved to `/scratch/$USER/VASP-Flow` (see its `MOVED_TO_SCRATCH.txt`); work there.

20.3. Quick OUTCAR checks

```
grep "reached required accuracy"   OUTCAR           # converged?
grep "energy  without"             OUTCAR | tail -1 # total energy
grep "E-fermi"                     OUTCAR | tail -1 # Fermi level
grep "magnetization (x)"           OUTCAR | tail -5 # moment
grep "TOTAL-FORCE"                 OUTCAR | tail -20
grep -A3 "BORN EFFECTIVE CHARGES"  OUTCAR | head -20
```

Appendices: how it works

A. How the inputs are generated

A.1. k-mesh formula

kpra (k-points per reciprocal atom) $= N_x N_y N_z \times N_{\text{atoms}}$. Given a target kpra and the reciprocal lattice vectors $\mathbf{b}_i^*$ (columns of $2\pi(\mathbf{A}^{-1})^\top$), the mesh follows from one formula, with no search:

$$N_i = \text{round}_2\left(s\,|\mathbf{b}_i^*|\right), \qquad s = \left(\frac{N_k}{|\mathbf{b}_1^*||\mathbf{b}_2^*||\mathbf{b}_3^*|}\right)^{1/3}, \quad N_k = \text{kpra}/N_{\text{atoms}},$$

where round$_2$ rounds to the nearest *even* integer (minimum 2), so that BZ boundary points ($k = 1/2$) are on the mesh. Because every axis uses the same s, the spacing $|\mathbf{b}_i^*|/N_i$ is the same along all three axes up to that rounding (a uniform mesh), axes with equal $|\mathbf{b}_i^*|$ get equal N_i, and $N_1N_2N_3N_{\text{atoms}} \approx$ kpra. For hexagonal lattices ($a = b$, $\gamma = 120°$, detected from the POSCAR) the two in-plane N are rounded to multiples of 6 ($6, 12, 18, \ldots$) so that K and M lie on the mesh. For 2-D slabs the same formula is used in the plane (square root instead of cube root) and $N_z = 1$, so the in-plane mesh carries the whole target. The first line of each generated KPOINTS reports the mesh and the kpra actually reached. Any integer may be given instead of `coarse`/`fine` (`KMESH_DENSITY: 3000`); the relaxation has its own density (`RELAX_KMESH_DENSITY`, default coarse).

A.2. KPAR and NCORE rule

When KPAR/NCORE are not specified they are set *per step* from two numbers only: the total MPI rank count N and that step's irreducible k-point count n_k (the spglib reduction of the step's own KPOINTS; the LOBSTER NSCF runs `ISYM=0`, so its full mesh counts).

Quantity	Rule
KPAR	the largest divisor of N that is $\leq n_k$. k-point groups scale almost perfectly, so use as many as there are k-points; a divisor, so no rank is idle.
NCORE	the largest divisor of $r = N/\text{KPAR}$ (ranks per k-group) that is $\leq \sqrt{r}$, VASP's own recommendation (NCORE = 1 for 1–3 ranks per group).

Examples with $N = 40$: $n_k \geq 40 \Rightarrow$ KPAR = 40, NCORE = 1; $n_k = 20 \Rightarrow 20/1$; $n_k = 7 \Rightarrow 5/2$; $n_k = 1 \Rightarrow 1/5$. If only one of the pair is given (instruction key, per-step key or an `INCAR` block), it is kept — snapped down to a divisor of N — and the other is derived from it: a fixed KPAR gets NCORE from the rule above; a fixed NCORE gets the largest KPAR $\leq n_k$ that leaves a multiple of NCORE ranks per group. Priority, highest first: ELF pin (KPAR = 1) > `INCAR` block > per-step key (`SCF_KPAR`, ...) > global `KPAR`/`NCORE` > the rule.

Three steps opt out: **DFPT** is always KPAR = NCORE = 1 (linear response supports neither); **phonons** and **Wannier90** default to KPAR = 1 (independent supercell jobs / the Wannier interface), with NCORE from the rule. With 40 ranks and the dense meshes of a small cell ($\leq$20 atoms, hundreds of irreducible k-points), KPAR = 40, NCORE = 1 is therefore the expected, correct result; it costs one copy of the wavefunctions per rank, which is small for such cells.

> **Note.** `LELF=.TRUE.` requires KPAR = 1. With `ELF: on` (the default) the SCF itself runs LELF and is pinned to KPAR = 1. With `ELF: separate` the SCF keeps the automatic KPAR and a short KPAR = 1 restart writes the ELFCAR afterwards (Section A.7). Convergence tests drop `LELF` automatically.

A.3. Eigensolver (ALGO) of each step

Relaxation and SCF use `ALGO = Normal` (blocked Davidson), the robust choice while the density still changes. The fixed-density steps — bands, DOS and the LOBSTER NSCF (`ICHARG = 11`) — use `ALGO = Fast`: a few Davidson steps, then RMM-DIIS. With the density fixed, pure Davidson ran away to $|E| \sim 10^{10}$ eV for Ba_2YReO_6 and Sr_2YReO_6 (independent of KPAR/NCORE); `ALGO = Fast` converged in 20 steps and `ALGO = VeryFast` (RMM-DIIS only) also converged, while `ALGO = All` (conjugate gradients) is not allowed with `ICHARG = 11` and stops VASP. A functional's own ALGO (HSE06: `Damped`) and any INCAR block win over these defaults. `vasp_diagnose.py` repairs a diverged or unconverged fixed-density step along `Normal` → `Fast` → `VeryFast`, and a non-converging SCF with `ALGO = All`.

A.4. LOBSTER k-mesh and memory

The LOBSTER NSCF must run without symmetry (`ISYM = 0`), so VASP keeps every k-point of the mesh; only time reversal pairs $\mathbf{k}$ with $-\mathbf{k}$:

$$N_\mathrm{k} = \frac{N_1 N_2 N_3 - t}{2} + t, \qquad t = 2^{\#\{i:\ N_i \text{ even}\}}$$

(6916 for 24×24×24, 2820 for 16×16×22, as VASP reports). Its memory grows with $N_\mathrm{k} \times N_\mathrm{PW}$, where the plane waves per k-point follow from the cell volume and the cutoff, $N_\mathrm{PW} = V k_c^3/(6\pi^2)$ with $k_c = \sqrt{\mathrm{ENCUT}/3.81\ \mathrm{eV\,Å^2}}$. Two runs with $N_\mathrm{k} N_\mathrm{PW} = 1.5 \times 10^7$ and 1.8×10^7 stopped at set-up with `insufficient virtual memory`, on two 96-core nodes as well as on one 40-core node with 375 GB — more memory per rank did not help, a smaller mesh does. The generators therefore start from the SCF mesh and, if it has more than `LOBSTER_MAX_KPTS` points ($N_1 N_2 N_3$, default 10000) or $N_\mathrm{k} N_\mathrm{PW} >$ `LOBSTER_MAX_KPW` (default 6×10^6), scale all N_i by the same factor (even, at least 2; $N_z = 1$ for slabs; in-plane multiples of 6 for hexagonal cells) until both hold. The point limit alone would not be enough: 16×16×22 has only 5632 points but failed, because that cell has twice as many plane waves per k-point. The KPOINTS comment then reads e.g. `reduced from 24x24x24: 2052 k x ~2822 PW`. The KPAR rule uses the reduced count. Set either limit to 0 (`LOBSTER_MAX_KPTS: 0`, `--lobster-max-kpw 0`) to switch it off, or a larger `LOBSTER_MAX_KPW` on nodes with more memory. For COHP/ICOHP integrals a mesh of a few hundred to two thousand k-points is ample for cells of up to ∼20 atoms.

A.5. Relaxation → SCF hand-off

The relaxation starts from scratch (`ISTART = 0`, `ICHARG = 2`) and writes `WAVECAR` and `CHGCAR`. `02_scf/copy_from_relax.sh` copies the relaxed geometry and the CHGCAR; if the SCF k-mesh, spin and cutoff equal the relaxation's it also copies the WAVECAR and the SCF starts from both (`ISTART = 1`, `ICHARG = 1`), otherwise it starts from the CHGCAR alone (`ISTART = 0`, `ICHARG = 1`) because wavefunctions on different k-meshes are not interchangeable. The decision is made at run time, so k-meshes edited or patched after generation are handled. Putting `ISTART` or `ICHARG` in an `INCAR scf:` block disables the mechanism.

A.6. Band-path detection

Since v1.6 the generator first determines the *actual space group* of the POSCAR with spglib (via pymatgen's `KPathSetyawanCurtarolo`) and uses the standard Setyawan–Curtarolo high-symmetry path for that space group. Every high-symmetry point is transformed through Cartesian reciprocal space into the reciprocal basis of *this* POSCAR, so the path is correct for the actual cell used in the SCF regardless of orientation or setting.

If spglib or pymatgen is not installed (or the symmetry analysis fails), the generator falls back to the geometric classifier below: it reads the POSCAR lattice vectors, computes cell lengths and angles, and selects a default path by Bravais type. 2-D structures always use the geometric route so the in-plane path is preserved.

Type	Detection criterion	Default path
cF (FCC)	a=b=c, angles ≈ 60°	G-X-W-K-G-L-U-W
cI (BCC)	a=b=c, angles ≈ 109.5°	G-H-N-G-P-H
cP (cubic)	a=b=c, 90°	G-X-M-G-R-X
hP (hexagonal)	a≈b, γ=120°	G-M-K-G-A-L-H-A
tP (tetragonal)	a≈b≠c, 90°	G-X-M-G-Z-R-A-Z
oP (orthorhombic)	90°, a≠b≠c	G-X-S-Y-G-Z-U-R-T-Z
rP (rhombohedral)	a=b=c, equal angles	G-T-F-G-L
mP (monoclinic)	α=γ=90°, β≠90°	G-Y-A-Z-G-B-D

A.7. Two-step ELF (`ELF: separate`)

`LELF` only works with KPAR = 1, which removes the k-point parallelism that makes an SCF on a small cell fast. With

```
ELF: separate          # or  ELF: two-step
```

the ELF is produced in two steps inside `02_scf`, in the same job:

1. The SCF runs normally — automatic KPAR/NCORE, no `LELF`, `LWAVE/LCHARG=.TRUE.`.
2. `run_elf.sh` (called by `run.sh` after VASP) copies the converged `WAVECAR`, `CHGCAR`, `POSCAR`, `KPOINTS` and `POTCAR` into `02_scf/elf/` and runs VASP there with `02_scf/elf/INCAR`: the SCF INCAR (same ENCUT, smearing, MAGMOM, $+U$, user overrides) plus `ISTART=1`, `ICHARG=1`, `LELF=.TRUE.`, `LWAVE/LCHARG=.FALSE.`, KPAR = 1 and NCORE from the rule for one k-group. `NBANDS` is set at run time to the value the SCF used (read from its `OUTCAR`), so the WAVECAR is read as is. Starting from converged wavefunctions, this restart needs only a few electronic steps.
3. `ELFCAR` is copied back to `02_scf/ELFCAR`, where `analyze.sh`, `elf_bonds.py` and the GUI look for it; the copied WAVECAR/CHGCAR in `elf/` are deleted.

If the SCF left no WAVECAR/CHGCAR, the ELF pass is skipped with a message. The DOS, bands and LOBSTER steps still read `02_scf/CHGCAR`, which the ELF pass does not touch. `ht-mp-scf.py` uses this mode by default; write `ELF: on` in `instructions.txt` for the old single run.

A.8. Convergence tests: test structure and parameter choice

A.8.1. Shifted test structure

The POSCAR used in `00_convergence` has **atom 1 displaced by** (0.01, 0.02, 0.03) **Å (Cartesian)** so that the force on it is non-zero and its convergence can be tracked — at the equilibrium (high-symmetry) positions the forces would vanish identically and the force panel would be meaningless. The displacement is converted through the lattice for Direct-mode POSCARs. All production steps (`01_relax`, `02_scf`, ...) use the *original, unshifted* positions.

A.8.2. Automatic parameter selection

After the runs, `00_convergence/choose_params.py` (invoked by `run_convergence.sh`, `analyze_convergence.sh`, and the phase-2 scripts) selects the smallest ENCUT and k-mesh from which every subsequent change stays below

$$\max_i |\Delta F_i^{\text{atom 1}}| < 5 \text{ meV/Å} \quad \text{and} \quad \max_i |\Delta \sigma_{ii}| < 0.5 \text{ kbar},$$

and writes the result to `00_convergence/converged_params.txt`. Phase 2 (`run_calculations.sh`, `submit_calculations.sh`, and the GUI form) offers these values as defaults; when run *non-interactively* — e.g. a convergence test automatically followed by the SCF in a chained/nohup run — they are applied without prompting. If the criteria are not met within the tested range, the largest tested value is used and a `NOT CONVERGED` warning is issued. Thresholds can be overridden: `choose_params.py -ftol 3 -ptol 0.2`.

A.9. DFPT: fixed parallel and symmetry settings

> **Note.** Both routes support neither k-point nor band parallelisation: **KPAR and NCORE are forced to 1** in the generated INCAR (overriding any global or per-step setting). The full MPI task count is still used for plane-wave parallelisation. Likewise **ISYM=2 is enforced** (symmetry imposed): perturbation theory relies on the symmetry-reduced perturbation set, so a symmetry-off `ISYM=0/−1` — e.g. from a global `INCAR:` override block intended for the LOBSTER/NSCF steps — is never inherited by `06_dfpt`.

`copy_from_scf.sh` (invoked by `run.sh`) pulls `WAVECAR` and `CHGCAR` from `02_scf` at run time. If `02_scf` has been converted in place for LOBSTER (`ISYM=0/−1` in its INCAR, as done by the high-throughput `run_scf_for_lobster.sh`), the `WAVECAR` copy is skipped — it holds the full, unsymmetrised k-mesh, which the ISYM=2 run cannot restart from — and VASP regenerates the orbitals from the `CHGCAR`, which the LOBSTER conversion (a fixed-charge NSCF with `LCHARG=.FALSE.`) leaves untouched. If the symmetry-off INCAR is *not* a fixed-charge NSCF, a warning is printed that the `CHGCAR` itself may stem from a symmetry-off SCF and a symmetric SCF should be re-run first.

A.10. LOBSTER projected DOS vs. the VASP DOS

The LOBSTER card also carries a projected-DOS plot (`analysis/*_lobster_dos.{png,pdf}`, generated by every interface but displayed only in the GUI), drawn in the same cumulative-stacked layout as the `04_dos` projected DOS. It answers a question the two DOS files raise on their own.

VASP's `LORBIT=11` PDOS projects onto spherical harmonics *inside the PAW augmentation spheres*, which do not fill space, so the interstitial charge is unassigned and $\sum$PDOS $<$ total DOS — severely so for free-electron-like metals (only 25% of the electrons for fcc Al; 72% for $MoSe_2$, whose Mo-*d* and Se-*p* states are compact). LOBSTER instead re-expands the PAW states in a local atomic-orbital basis and performs a Mulliken population analysis, so its decomposition is complete by construction: the stacked areas sum *exactly* to LOBSTER's own total.

The plot therefore shows three things: the stacked LOBSTER channels, LOBSTER's total (dashed grey, identically the top of the stack), and the projection-free VASP total from `04_dos` (solid black). The gap between the last two is the charge spilling, reported in `lobsterout` and typically 1–4%. For the *total* DOS the VASP curve is the reference — it is a direct eigenvalue count under the tetrahedron method, with no projection at all – whereas for *orbital-resolved* weights the LOBSTER decomposition is the more complete one, and is the same basis that produces the COHP/COBI curves. Note that the two curves also carry different broadening (`04_dos` uses `ISMEAR=-5`; the LOBSTER NSCF is forced to Gaussian smearing, and LOBSTER applies its own `gaussianSmearingWidth` on a coarser energy grid), so some of the visible difference is resolution rather than physics. Both smearing settings are stated in the plot legend.

B. How SLURM steps continue past the time limit

A step that reaches its limit is continued, not lost. Every step script asks SLURM for a warning 15 min before the limit (`--signal=B:USR1@900`). On the warning it writes a `STOPCAR` (`LSTOP` for a relaxation: stop after the current ionic step; `LABORT` otherwise), lets VASP finish cleanly and requeues the job with `scontrol requeue` — same job id, so jobs chained after it with `afterok` keep waiting. The requeued job then

- **relaxation**: continues from the `CONTCAR` (or, if that is empty, from the last `XDATCAR` frame, cell included);

- **LOBSTER**: skips the NSCF if it had finished and reruns only the LOBSTER binary;
- other steps: rerun.

The interrupted `OUTCAR`/`XDATCAR` are kept as `*.timeout0`, `*.timeout1`, ... If requeueing is not allowed on the cluster, the job resubmits a copy of itself with the same name and time limit and re-points dependent jobs to it. At most `VF_MAX_RESTARTS` (default 5) continuations per step.

C. High-throughput drivers: all options

C.1. `ht-mp-scf.py` in detail

For each ID it stages `_ht_inputs/<id>/{POSCAR, instructions.txt}` and writes `runall.sh`, which calls `vasp-agent.py` (local) or `vasp-agent-slurm.py` (SLURM) to build the inputs and then runs or submits each material. `runall.sh` downloads nothing.

Relaxation first. `01_relax` relaxes cell shape, volume and ions (`IBRION = 1`, `ISIF = 3`), because the MP cell was relaxed with a different functional/setup. It uses the SCF k-point density (`--relax-kpra` to change it), so the SCF starts from the relaxed `CONTCAR`, `CHGCAR` and `WAVECAR` (`copy_from_relax.sh`).

ELF in two steps. The instructions carry `ELF: separate`: the SCF runs with the automatic KPAR/NCORE, then a short KPAR = 1 restart in `02_scf/elf/` writes `ELFCAR`, copied to `02_scf/ELFCAR` (Section A.7).

k-points. Uniform mesh at `--kpra` (default `coarse` = 1000); per material via `KMESH_DENSITY` in `_ht_inputs/<id>/instructions.txt`; `--kmesh` forces one fixed grid for all (not recommended for mixed cell shapes).

Option	Meaning
`-l`, `--list`	ID file (default `highthrouput_list` in the current directory).
`--agent`	`local` or `slurm`; prompts if omitted.
`--poscar-dir DIR`	Reuse downloaded POSCARs (see above).
`--chain` / `--no-chain`	SLURM: chain every step of every material with `afterok` (default), or submit each material independently (its own relax → SCF → LOBSTER chain). Local runs are always sequential.
`--relax` / `--no-relax`	Relaxation first (default on).
`--relax-kpra`	k-density of the relaxation (default: `--kpra`).
`--gga_u auto|on|off`	GGA+U flag (default `auto`: only chalcogenides/halides).
`--kpra`	`coarse`, `fine` (6000) or an integer.
`--kmesh`	Fixed Γ-mesh, e.g. `'8 8 8'`; overrides `--kpra`.
`--encut`	ENCUT (eV); default 500.
`--functional`	PBE (default), PBEsol, R2SCAN, ...
`--lobster` / `--no-lobster`	LOBSTER step (default on).
`--kpar`, `--ncore`	Force KPAR/NCORE (give one, the other is derived); the ELF pass always uses KPAR = 1.
`--mpi`	MPI ranks per job (workstation).
`--walltime`	One walltime for every step (default: per-step defaults, Section 11.4).
`--nodes`, `--ntasks-per-node`, `--partition`, `--account`, `--profile`	SLURM resources, written into each `instructions.txt` (they override `profiles/slurm.json`).

Output	Location
Material folders	`<mp-id>/01_relax, 02_scf, 08_lobster, analysis`
ELFCAR	`<mp-id>/02_scf/ELFCAR` (ELF pass in `02_scf/elf/`)
Staged inputs	`_ht_inputs/<mp-id>/`
Element / Z table	`_ht_inputs/elements_Z.txt`

Note. A download error `'dict' object has no attribute 'number'` is an spglib/pymatgen version mismatch: `pip install -U spglib`. The driver falls back to `Structure.get_primitive_structure()` automatically.

C.2. `ht-semimetals.py` in detail

It builds a self-contained `highthroughput_*` directory: one `materials/<id>/job.sbatch` per material; `submit_all.sh` submits it once per step (`relax`, `scf`, `bands`, `lobster`) as an `afterok` chain with the walltimes `TIME_RELAX/TIME_SCF/TIME_BANDS/TIME_LOBSTER` of `env.sh`, and steps that reach their limit continue automatically (Section 11.4). The steps are the relaxation (`01_relax`, IBRION = 1, ISIF = 3, same k-density as the SCF; if the first pass does not reach the force criterion it is restarted once from its CONTCAR), the SCF from the relaxed structure, bands and the LOBSTER NSCF + LOBSTER (finished steps are skipped on resubmission); the LOBSTER job ends and ends with `analyze.sh` (plots and `lobster_summary.csv` in `analysis/`). ENCUT and the LOBSTER NBANDS come from the POTCARs at generation time and are checked against `potcar_manifest.json` when the POTCARs are built on the cluster. POTCAR variants follow the Materials Project choices by default (`--potcar-set mp`: e.g. Re_pv, Ti_pv, Ga_d, Li_sv, taken from pymatgen's `MPRelaxSet`); the older "plain element first" rule (`--potcar-set minimal`) gave minimal-valence POTCARs, and Ba_2YReO_6/Sr_2YReO_6 with plain Re collapsed into an unphysical state. Relax, SCF and bands use the same `NBANDS` = 1.2× VASP's default (from NELECT, NIONS and the starting moment; rounded up to even): VASP's default leaves too few empty bands for narrow d bands, the electron count then drifts between SCF steps (`BRMIX: very serious problems`) and the SCF diverges. The bands step never uses fewer bands than its SCF. The k-point density is `--kpra` (default 8000 k-points per reciprocal atom), the same spacing along all axes. The LOBSTER NSCF uses the SCF mesh, reduced when it exceeds `--lobster-max-kpts` or `--lobster-max-kpw` (Appendix A.4). Every job requests the whole node's memory (`#SBATCH --mem=0`). `--kspacing` is an alternative to `--kpra`; before v2.5 it was the default (0.10 $Å^{-1}$, very dense: 24×24×24 for the 8-atom $Sc_2Al_2C_2O_2$, 110592 kpra). 3d elements (V–Ni) start spin-polarised. GGA+U follows the default rule (only chalcogenides/halides, Section 6.4) unless `--gga_u on|off` is given. There is no DFPT step (dielectric tensors and Born charges are ill-defined at zero gap, and the linear-response loop tends not to converge) and no ELF (`ELF: off`).

Option	Meaning
`--list`, `--out`	ID file and directory to create.
`--kpra N`	k-points per reciprocal atom (default 8000; 1000 = coarse).
`--kspacing D`	instead of `--kpra`: fixed spacing, $N_i = \text{even}\lceil \lvert\mathbf{b}_i^*\rvert / D \rceil$ (0.10 Å^{-1} is very dense).
`--lobster-max-kpts N`	largest $N_1 N_2 N_3$ of the LOBSTER mesh (default 10000, 0 = no cap).
`--lobster-max-kpw X`	LOBSTER memory budget, k-points × plane waves (default 6×10^6, 0 = no cap).
`--relax` / `--no-relax`	Relaxation first (default on).
`--gga_u auto|on|off`	GGA+U flag written into every `instructions.txt` (default `auto`: only chalcogenides/halides).
`--potcar-set mp|minimal`	POTCAR variants: Materials Project choices (default) or plain element first, then _sv/_pv/_d.
`--single-node`	Every material on one node with `--cores-per-node` (default 40) ranks; otherwise large cells get more nodes.
`--kpar`, `--ncore`	Force KPAR/NCORE (default automatic).
`--max-atoms`, `--max-nodes`	Skip larger cells / node limit.
`--build-potcars` / `--no-build-potcars`	Build POTCARs right after generation (default: when `sbatch` exists).
`--include-potcars`	Ship assembled POTCARs in the directory.
`--poscar-dir DIR`	Reuse downloaded POSCARs (see above).
`--partition`, `--account`, `--modules`, `--jobs N`	Prepare N materials in parallel (default min(8, CPUs)).
`--vasp-std`, `--lobster-bin`, `--hpc-potcar-dir`	Cluster settings, written into `env.sh` and the job scripts.

D. How `vasp_diagnose.py` works

D.1. Command line

```
python3 vasp_diagnose.py LIST [options]
```

Option	Default	Meaning
`--out DIR`	`vasp_diagnose_<date>_<time>`	folder for the report and the scripts
`--root DIR`	—	another folder to look up material names in (can be repeated)
`--since DATE`	14 days ago	how far back SLURM's job records (`sacct`) are read, YYYY-MM-DD; must reach back to the jobs being diagnosed
`--kpar-max N`	8	largest KPAR written by the out-of-memory repair
`--spill-max X`	5	LOBSTER charge spilling (%) above which a material is flagged
`--retry-unknown`	off	also submit the materials whose failure was not recognised
`--max-queued N`	200	default queue limit of `submit_all_fixes.sh` (override later with `MAX_QUEUED=`)

It needs only Python ≥ 3.6 (no extra packages) and uses `sacct` and `squeue` when they exist.

D.2. The list file

One entry per line; empty lines, `#` comments and anything after the first word are ignored. An entry is

Entry	Checked
a **material name** — any folder name: `mp-149`, `GaAs`, `Bi2Se3_soc`, …	the folders of that name in R, R`/*/` or R`/*/materials/`, where R is the folder holding the list file, and each `--root`
a **material or project folder** (has `00_convergence`, `01_relax`, …)	that folder
a **run folder** with `materials/`	every material in `materials/`; if the run has a `material_list.txt`, only the materials in it (other folders are named and skipped)
any **other folder**	every sub-folder that is a material/project folder

A word that is a folder (relative to the list file's folder) is taken as a path; otherwise it is a material name. Names are matched exactly, so `GaAs` finds `.../GaAs` but not `.../GaAs_PBE`, and a name may contain any characters a folder name can. A name that is found nowhere is reported as `missing`.

D.3. Kinds of runs

How the jobs are resubmitted depends on how the run was made:

Layout	Recognised by	Resubmitted with
`ht`	`job.sbatch` in the material folder (high-throughput drivers)	`sbatch job.sbatch <step>`, walltimes from `TIME_*` in the run's `env.sh`; steps relax, scf, bands, lobster
`slurm`	`run.sh` files with `#SBATCH` lines (`vasp-agent-slurm.py`)	`sbatch <step>/run.sh`, with its job name and walltime
`local`	anything else (`vasp-agent.py`)	`(cd <step> && bash run.sh)`, in the foreground

Each material gets a *label*, `<run folder>__<material>` (e.g. `HT_semimetals__mp-149`, `projects__GaAs_test`); it names the repair script and the report line.

D.4. When is a step OK?

Step	OK when
`00_convergence`	every point of the ENCUT test (`encut/encut_<E>/`) and of the k-point test (`kpoints/<N1>x<N2>x<N3>/`) that its `run.sh` will make has finished with a converged SCF. If `converged_params.txt` says a quantity did not converge in the tested range, this is noted in the report (widen the range yourself)
`01_relax`	OUTCAR has `reached required accuracy`
`02_scf`	OUTCAR finished (`General timing and accounting`) and the SCF converged (`aborting loop because EDIFF is reached`)
`03_bands`	finished, converged, and no energy above 10^6 eV in OSZICAR
`04_dos`	as bands, and a non-empty DOSCAR
`06_dfpt`	finished with Born charges or the dielectric tensor in OUTCAR
`08_lobster`	`lobster.out` has `finished in`; a charge spilling above `--spill-max` is reported but counts as OK

`05_wannier` and `07_phonons` are not checked (noted as `unchecked`).

For a step that is not OK, the cause is read from the step's newest SLURM logs, `vasp.out`, the end of OUTCAR and the SLURM job record (state, run time, time limit). The job record wins: a job SLURM ended for its time limit is a `timeout`, whatever else is in the logs. So is a VASP run stopped by the STOPCAR that the VASP-Flow scripts write shortly before the time limit (`hard stop encountered`, `soft stop encountered`). Messages after which VASP carries on, such as `Sub-Space-Matrix is not hermitian` or `The number of bands has changed`, are shown in the script header but never taken as the cause.

D.5. Failure causes and their repairs

Name	Meaning	Repair
`oom`	out of memory (`insufficient virtual memory`, `Cannot allocate memory`, killed by the OOM handler, MPI time-outs between nodes)	KPAR lowered to at most `--kpar-max` (a divisor of the MPI ranks); all node memory requested; for the LOBSTER NSCF also its k-mesh halved per axis
`timeout`	the job hit its time limit	walltime doubled; a relax continues from its last structure
`scf_nelm`	SCF stopped at NELM without converging	`ALGO = All`, `NELM` ≥ 150
`nscf_diverged`	bands/DOS/LOBSTER energies exploded	30% more bands than the SCF (not for LOBSTER); eigensolver `Normal` (Davidson, VASP's default) $\rightarrow$ `Fast` $\rightarrow$ `VeryFast` (RMM-DIIS), then held; `NELM` ≥ 150
`nscf_unconv`	bands/DOS stopped at NELM	`ALGO` as for nscf_diverged, `NELM` ≥ 150
`relax_unconv`	relax ended without reaching the force criterion	continues from CONTCAR (or the last XDATCAR frame)
`zbrent`	`ZBRENT: fatal error` in a relax	`IBRION` 2 $\rightarrow$ 1, continue
`symmetry`	VASP symmetry errors	`SYMPREC = 1E-4` (`ISYM = 0` if SYMPREC was set)
`edddav`	`EDDDAV`/`ZHEGV` errors	SCF: `ALGO = All`; relax: `ALGO = Normal`, if that fails again `ALGO = All`; bands etc.: as nscf_diverged
`wavecar`	WAVECAR does not fit the new run	WAVECAR moved aside, start from CHGCAR
`chgcar`	CHGCAR unreadable or cut off	SCF started from scratch (`ICHARG = 2`)
`chgcar_missing`	a later step needs `02_scf/CHGCAR`, which is missing or cut off	SCF redone first
`lobster_fail`	VASP part of the LOBSTER step finished, the LOBSTER program did not	only the LOBSTER program is rerun
`quota`	disk full / quota exceeded	resubmitted unchanged — **free space first**
`mpi_init`	MPI failed to start (bad node)	resubmitted unchanged
`not_run`	never ran (e.g. an earlier step failed)	submitted
`unknown`	none of the above	script written but held back for review
(any) `...` `(fix already in INCAR)`	the repair for this cause is already in the INCAR, so it did not help	script written but held back for review
`spilling_high`	LOBSTER finished, spilling > `--spill-max`	none (report only)
`missing`	a listed name has no folder	none (report only)
`phase2_pending`	convergence tests done, production steps not started	none: start them with `submit_calculations.sh` / `run_calculations.sh`
`queued`	the material still has jobs in the queue	none: diagnose later

A material is counted under the cause of its *first* failed step. A charge spilling is a property of

the LOBSTER basis set, so rerunning does not change it; lowering it needs other basis functions in `lobsterin`.

D.6. What is redone

- If the relax or the SCF failed, that step and *all* later steps are redone.
- Otherwise only the failed later steps (bands, DOS, DFPT, LOBSTER). If they need `02_scf/CHGCAR` and it is missing or cut off, the SCF is redone first.
- **Convergence tests** are repaired on their own: only the broken points are moved aside, the fix of their cause goes into the test's INCAR, and the test's `run.sh` is resubmitted — it skips the points that are already done. Production steps are not redone because of a convergence test.
- **LOBSTER only**: when the VASP part of the LOBSTER step finished and its WAVECAR is there, only the LOBSTER program reruns (not for local projects).
- In high-throughput runs `job.sbatch` cannot run DOS or DFPT; such steps are reported (`job.sbatch cannot run: dos`) but not resubmitted.

What a repair script changes in a material folder:

- **INCAR**: every line of a changed tag is removed and `TAG = value` is added at the end. No other tags are touched.
- **Old outputs** of every redone step (OUTCAR, OSZICAR, CHGCAR, WAVECAR, `*.lobster`, ..., or the whole broken convergence point) are moved to `<folder>/stale_<date>/`. Inputs stay in place; nothing is deleted.
- **Relax restarts** from the newest complete CONTCAR (else the last XDATCAR frame); the original POSCAR is kept once as `POSCAR.orig`.
- **SCF start**: if the relax's CHGCAR is cut off, the SCF starts from scratch.

D.7. Order of the resubmitted jobs

Layout	Order
`ht`	each step waits for the previous one to succeed (`afterok`); LOBSTER after bands waits only for it to end (`afterany`), as it does not need the bands
`slurm`	SCF waits for the relax; bands, DOS, DFPT and LOBSTER wait for the SCF
`local`	one after the other; stops at the first failure

Convergence tests are submitted without waiting for anything. Walltimes are the run's own (`env.sh`, `#SBATCH --time`, or relax 8 h, SCF 4 h, bands/DOS 2 h, DFPT 8 h, LOBSTER 4 h, tests 8 h), doubled after a time-out.

D.8. Output files

`diagnosis.tsv` — tab-separated, one line per material:

Column	Content
`label`	the material's label
`layout`	`ht`, `slurm`, `local` (`-` for a missing name)
`path`	its folder
`steps`	`step:status` for each step (`conv:encut`, `conv:kpoints`, `relax`, ...) and notes
`action`	`ok`, a report-only name, or `<cause> -> <steps redone>`; `REVIEW` in front when held back
`script`	its repair script, if any

```
awk -F'\t' 'NR>1␣{split($5,a,"␣");␣n[a[1]]++}␣END␣{for␣(k␣in␣n)␣
    print␣k,␣n[k]}' diagnosis.tsv
awk -F'\t' '$5␣~␣/^REVIEW/␣{print␣$1}' diagnosis.tsv
```

`fix/<label>.sh` — a stand-alone bash script; it can be run on its own. In order it:

1. records the diagnosis and the error lines in its header;
2. **stops** if the material has jobs in the queue, or if any of its OUTCARs or `lobster.out` changed since the diagnosis (so it is safe to run late, or twice);
3. applies the INCAR changes and moves the old outputs aside;
4. submits the jobs (or runs them, for a local project);
5. leaves a marker `.submitted/<label>` in the output folder.

`submit_all_fixes.sh` — runs every repair script in turn; skips those with a `.submitted` marker; waits while you have `MAX_QUEUED` or more jobs in the queue (all your jobs count); lists the held-back `unknown` scripts at the end as commented-out lines.

D.9. Limitations

- Jobs are recognised by their job names (`<material>_<step>`, or the name in the step's `run.sh`). Jobs submitted under other names are not seen, and a failure cause is then taken from the log files only.
- Only jobs since `--since` are known to the tool. Without a job record a time-out is still recognised from the log, but the walltime is doubled from the configured value.
- A second out-of-memory failure with KPAR already $\leq$ `--kpar-max` is resubmitted with the same KPAR (the LOBSTER mesh is halved again); diagnose again with a smaller `--kpar-max`.
- A material that fails again with the same cause after one repair (e.g. `scf_nelm`) needs a look by hand.
- `quota` and `mpi_init` are resubmitted unchanged: free disk space first; node failures are usually transient.
- `05_wannier` and `07_phonons` are not checked; a convergence-test range that was too small is reported but not widened.
- Two runs holding the same material use the same job names (`<material>_<step>`), so a job of one makes the material show as `queued` in both.
- Repair scripts use GNU `sed` and `stat`: they are meant for Linux clusters.

E. Version history

v1.0 (2026-03-23) Initial release — GUI, workstation agent, PBEsol/R2SCAN/HSE06, SOC, GGA+U, convergence tests, Wannier90, phonopy.

v1.1 (2026-04–05) Per-task KPAR/NCORE smart defaults; SLURM agent with dependency-chained submission; cumulative DOS by element (no pymatgen); copy-from-relax/scf scripts.

v1.2 (2026-06) Collinear magnetism: MAGMOM per atom, spin-resolved band and DOS plots. Auto k-path detection (8 crystal systems). Fixed DOS projection parsing. Separate GUI and agent quick-reference sheets. Rewritten comprehensive LaTeX manual.

v1.3 (2026-06-15) Raw INCAR passthrough (`INCAR: ... END_INCAR`, global or per-step) for arbitrary VASP tags. Constant-pressure relaxation via `PRESSURE` (ISIF=3, IBRION=2, PSTRESS).

v1.4 (2026-06-19) High-throughput driver `ht-mp-scf.py`: Materials Project primitive cells → batch SCF + ELF, with per-material element/Z table and cross-material `afterok` chaining on SLURM.

v1.5 (2026-06-24) Robust band plotting across the GUI and both CLI agents (workstation + SLURM): the built-in `vasprun.xml` plotter now handles non-spin as well as spin, and is used automatically when `sumo-bandplot` crashes or is unavailable. Fixed the matplotlib fallback (`plot_results.py`) to resolve the real Fermi level and emit PNG, not just PDF.

v1.6 (2026-07-08) Band k-path derived from the real space group via spglib (Setyawan–Curtarolo) in all agents and the GUI, with the geometric classifier kept as fallback. New `08_lobster` step: LOBSTER COHP/COBI/COOP bonding analysis with GUI plots and the `lobster_postprocess.py` aggregator; the NSCF uses ISYM=0, or ISYM=−1 when SOC is on. Full GPL-3.0 text in `LICENSE` (GitHub now detects the license correctly). Manual and quick-reference sheets brought up to date.

v1.7 (2026-07-08) DFPT step overhaul: KPAR/NCORE forced to 1 (linear response and PEAD support neither); automatic finite-field route (`LCALCEPS`, IBRION=6) when SOC, HSE06 or R2SCAN is requested; `extract_born.py` now reports ε_∞, the ionic contribution, total static ε_0, an acoustic-sum-rule check, element-labelled Z^*, and clamped-/relaxed-ion piezoelectric tensors.

v1.8 (2026-07-09) ELF bond profiles: `elf_bonds.py` plots the electron localization function along non-equivalent nearest-neighbour bonds (atom centre to atom centre), wired into `analyze.sh` for both CLI agents and shown in the GUI Results tab whenever `02_scf/ELFCAR` exists. Output is named after the reduced formula (`MoSe2_elf_bonds.pdf`). Detects the all-zero `ELFCAR` that VASP writes for non-collinear / spin-orbit runs and reports it instead of emitting a blank plot.

v1.9 (2026-07-22) *K-mesh and GGA+U.* Physics-based cell-aware k-mesh generation replacing fixed uniform grids. Two density tiers: **coarse** (1000 k-points per reciprocal atom, kpra) for convergence tests and relaxations; **fine** (5000 kpra) for SCF, DOS, and spectroscopic steps. Subdivisions $N_i \propto |\mathbf{b}_i^*|$ from reciprocal lattice vectors, all-even to include BZ boundary points; $N_z = 1$ for 2-D slabs. Hexagonal systems: fixed 6×6 (coarse) or 12×12 (fine) in-plane, N_z from $|\mathbf{b}_3^*|/|\mathbf{b}_1^*|$. DOS mesh always $2N_x \times 2N_y \times 2N_z$. DFPT forced to match SCF mesh exactly. New instruction key `KMESH_DENSITY` (coarse/fine). GGA+U anion gate extended to all chalcogenides and halides (O, F, S, Se, Te, Cl, Br, I); Hubbard lookup table now covers sulfides such as $AgCrP_2S_6$.

Plot improvements. ELF bond-profile and LOBSTER (COHP/COBI/COOP) plot legends and statistics boxes moved outside the axes area so they never obscure curves. Shared plotting module `modules/cohp_plot.py` used by both the GUI and the CLI agents; COHP, COBI, and COOP plots are now generated automatically in `analyze.sh` for both the workstation and SLURM agents, matching GUI behaviour.

Structure visualizer. New *Visualize* button in the POSCAR card: converts the current POSCAR to XYZ via `modules/poscar2xyz.py` and opens it in Jmol, VESTA, or any other viewer. The viewer path is set under System Setup → *Structure visualizer* (or `VISUALIZER_X` in `default_paths`).

Configuration. `LOBSTER_X` and `VISUALIZER_X` are now exposed in the GUI System Setup page and propagated to the agent subprocess environment. Platform config file renamed from `site.env` to `default_paths` for clarity; three-layer precedence (default_paths < GUI settings < profile JSON) documented. Auxiliary Python scripts (`band_plot.py`, `dos_plot.py`, `elf_bonds.py`, `lobster_postprocess.py`, `conv_choose.py`, `poscar2xyz.py`) consolidated into `modules/`.

v2.0 (2026-09-28) *Input-consistency release.* `MAGMOM:` now honours negative values, per-element (`Fe=4.0, O=0.6`) and per-atom lists, switches ISPIN on by itself, and initialises SOC runs along the requested x/y/z direction. K-mesh density is reflected in every KPOINTS file: coarse/fine/integer kpra, even $N_i \propto |\mathbf{b}_i^*|$ chosen for uniform spacing and closeness to the target, hexagonal in-plane meshes in multiples of 6, `RELAX_KMESH_DENSITY`; one generator routine (`kpoints_text`) serves both agents and the GUI. KPAR/NCORE: ELF-pinned KPAR=1 re-derives NCORE, and values given in INCAR blocks steer the companion value. Relaxation starts from scratch and writes WAVECAR/CHGCAR; the SCF reads CHGCAR, and WAVECAR too when the k-mesh matches. Explicit `INCAR <step>:` and new `KPOINTS <step>:` blocks override everything (GUI: Setup → Advanced). New `check_agent_consistency.py` compares GUI, workstation and SLURM outputs. New `ht-semimetals.py` builds the `highthroughput_*` SCF/bands/LOBSTER screening sets from list files.

v2.1 (2026-09-30) *Simpler k-mesh and parallel rules; two-step ELF.* k-mesh: one formula, N_i = nearest even (hexagonal in-plane: multiple of 6) of $s|\mathbf{b}_i^*|$, replacing the tolerance search. KPAR = largest divisor of the ranks $\leq n_k$; NCORE = largest divisor of the ranks per k-group $\leq$ its square root. Wannier90 and phonons now really default to KPAR = 1, and LOBSTER uses its own (ISYM=0) k-count. New `ELF: separate`: SCF at full KPAR, then a short KPAR = 1 restart writes ELFCAR (default in `ht-mp-scf.py`, which also gains `--kpra` (default coarse), `--kpar`, `--ncore`). `ht-semimetals.py`: `--kpar`/`--ncore`, POTCARs built automatically.

v2.2 (2026-09-30) *Relax-first high throughput; results viewer.* `ht-mp-scf.py`: `01_relax` (IBRION = 2, ISIF = 3) before the SCF by default (`--no-relax`, `--relax-kpra`); SLURM chaining now submits every step of a material, not only the SCF. `ht-semimetals.py`: DFPT step removed; each job runs `analyze.sh` at the end (`ANALYSIS_PYTHON` in `env.sh`), plus `analyze_all.sh`; `collect_results.sh` keeps the per-step layout. GUI: **Open results folder** views jobs run elsewhere and draws missing plots; SCF/NSCF steps show as done when VASP finished. NumPy 2 fix (`np.trapz` → `np.trapezoid`) in the COHP plot and LOBSTER post-processing. GGA+U flag `GGA_U: ON|OFF` in `instructions.txt` (GUI: auto/on/off; HT drivers: `--gga_u`); without it the tabulated U_{eff} is used only for chalcogenides/halides (O, S, Se, Te, F, Cl, Br, I), otherwise $U = 0$; same U in all steps; U values only from `hubbard_u_defaults.csv` (3d/4d/5d/4f/5f, `LDAUU` = U_{eff}, `LDAUJ` = 0). SLURM: walltime per step (relax 8 h, SCF 4 h, bands/DOS 2 h, LOBSTER 4 h); a

step reaching its limit is stopped cleanly and requeued (relaxations continue from CONTCAR/XDATCAR); `ht-semimetals.py` submits one job per step. `ht-semimetals.py` relaxes cell + ions first (`01_relax`, ISIF = 3).

v2.3 (2026-10-05) `ht-semimetals.py`: Materials Project POTCAR choices by default (`--potcar-set mp`); `NBANDS` = 1.2× VASP default in relax/SCF/bands, bands never below the SCF; `#SBATCH --mem=0`; relaxation with IBRION = 1. Maintenance tools in `ht_tools/` (`fix_failed.sh`, `test_relax.sh`, `collect_lobster.sh`, `reset_stale.sh`, `cleanup_large.sh`).

v2.4 (2026-10-12) *Checking and repairing runs.* New `vasp_diagnose.py`: checks the runs of a `*_list.txt` (any material or folder names; `00_convergence` to `08_lobster`; high-throughput, SLURM and local layouts), names the cause of each failure, writes a repair and resubmission script per material and `submit_all_fixes.sh`. Bands, DOS and the LOBSTER NSCF use `ALGO = Fast` (pure Davidson diverged at fixed density for Ba_2YReO_6/Sr_2YReO_6). `ht_tools/`: `test_step.sh` (test SCF/bands/LOBSTER settings on one material), `show_problems.sh`, `skip_fixes.sh`, `show_material_status.sh`, `show_job_history.sh`, `show_job_log.sh`. Manual reorganised by task (getting started, single calculations, high-throughput screens, checking and repairing runs; how things work in the appendices), with the `vasp_diagnose.py` manual merged in.

v2.5 (2026-10-12) k-meshes: `fine` = 6000 kpra (was 5000); `ht-semimetals.py` uses `--kpra 8000` by default (`--kspacing` only on request). The LOBSTER NSCF uses the SCF mesh (was 2×), kept within limits by default: with `ISYM = 0` the mesh is reduced (same ratios) above 10000 points (`LOBSTER_MAX_KPTS`) or when k-points × plane waves exceed `LOBSTER_MAX_KPW` (6×10^6); `vasp_diagnose.py` halves the LOBSTER mesh after an out-of-memory failure. `test_step.sh`: one-node tests go to the `standard` partition.